

Virly Frontend Component Reference

- Virly Frontend Component Reference
 - Conventions
 - Table of contents
 - Auth
 - Dashboard / Balance
 - Transfers
 - Transactions / History
 - AI Assistant
 - Layout / Navigation
 - Shared UI
 - Appendix A — Component index
 - Appendix B — Shared hooks, contexts, providers, and utilities
 - Contexts / providers
 - API client and types
 - Utilities
 - Appendix C — Design tokens / theme
 - 1. Tailwind v4 `@theme inline ( client/src/index.css )`
 - 2. Hand-written design system (`client/src/styles/global.css`)
- Auth
 - Components in this area
 - AuthLayout
 - AuthProvider
 - LoginPage
 - PersonalDetailsAuthForm
 - RegisterPage
 - ResendVerificationPage
 - SignInCard2
 - VerifyPage
- Dashboard / Balance
 - Components in this area
 - AccountStatement
 - DashboardPage
 - QuickContacts
- Transfers
 - Components in this area
 - EmptyRelationshipState
 - RecipientStatusCard
 - RelationshipSummaryCard
 - TransferCheque
 - TransferPage
 - TransferQuoteSmallPrint

- UserProfileHeader
 - UserProfilePage
- Transactions / History
 - Components in this area
 - RecentRelationshipTransactions
 - TransactionDetailsDialog
 - TransactionList
 - TransactionReceipt
 - TransactionsPage
- AI Assistant
 - Components in this area
 - AssistantBlocks
 - ChatWidgetDemo
 - FloatingChatWidget
- Layout / Navigation
 - Components in this area
 - App
 - AppShell
 - NotFoundPage
 - NotFoundSlip
 - RouteGuards
 - ShellTopbar
 - UserProfileSidebar
- Shared UI
 - Components in this area
 - AgentVideoSessionsPage
 - AnimatedText
 - Avatar
 - BentoCard
 - Button (shadcn/ui)
 - CurrencyProvider
 - CurrencySelector
 - JitsiMeeting
 - OrderConfirmationCard
 - Primitives (UI toolkit)
 - Select (shadcn/ui)
 - SettingsPage
 - ShaderBackground
 - VideoSessionPage

Virly Frontend Component Reference

Virly is a cash-transfer web application: a React + Vite + TypeScript frontend (this document), a Node/Express + MongoDB backend, and a LangGraph/OpenAI assistant layer behind that backend. This reference catalogues the frontend's visual components — what they render, the props they accept, the state and endpoints they touch, and the user flows they belong to. Read it area-by-area (Auth → Dashboard → Transfers → Transactions → AI Assistant → Layout/Navigation → Shared UI); each area is one Markdown file under `areas/`, and the consolidated PDF stitches them in that order.

Three architectural facts shape almost every component and recur in the **Architecture constraints** callouts:

1. **The backend is authoritative for all state.** The UI prepares and initiates actions; it never moves money or mutates a balance client-side. Balances shown in the shell come from the server and are refreshed only from server responses.
2. **Transfers require explicit confirmation.** A transfer executes only when the user confirms and the client calls an execution endpoint — `POST /api/transactions` for the manual cheque flow, or `POST /api/ai/confirmations/:id (action: "confirm")` for an assistant-prepared transfer. Everything before that point is preparation.
3. **The AI assistant cannot act on its own.** Its frontend is a read-only / preparation surface: it renders the server's structured output and surfaces prepared actions for the user to confirm. It cannot select tools or approve a transfer.

Screenshots are placeholders. Storybook capture is in progress separately. Every image reference already uses the final convention `images/<Component>--<state>.png`, and a matching placeholder PNG exists for each, so real captures drop in with zero Markdown changes. See [images/TODO.md](#) and [capture/capture-screenshots.ts](#).

Conventions

- **Frontend root:** `client/` (Vite 5 + React 18 + TypeScript, `react-router-dom v6`).
- **Styling:** Tailwind CSS v4 (`@theme inline` in `client/src/index.css`) for the shadcn-style `components/ui/*` set and the AI cards, plus a hand-written CSS design system in `client/src/styles/global.css` (BEM-ish classes like `.cheque`, `.statement`, `.signin-*`) for the bespoke “paper/ledger” surfaces.
- **Image naming:** `images/<ComponentName>--<state>.png` (e.g. `TransferCheque--error.png`). One image per meaningful state.
- **State vocabulary:** `default`, `loading`, `empty`, `error`, `success`, `disabled` (plus a few flow-specific states like `review` for the cheque). Components only document the states their source actually supports.

- **Props tables:**

Prop	Type	Required	Default	Description

. Types are taken verbatim from the source interfaces; “N/A” marks a field that does not apply rather than omitting it. Inferred details are labelled “(inferred)”.
- **Tiers:** **Lite** for primitives and simple layout (summary, screenshot, props, states, usage); **Full** for pages, features, forms, modals, providers, and containers (adds purpose, anatomy, state/data, interactions, dependencies, accessibility, related, gotchas).
- **Architecture constraints callout:** a blockquote on every Transfer and AI Assistant component restating the three facts above as they apply to that component — where the confirmation gate is, and that the component does not move money or let the assistant act unilaterally.

Table of contents

Auth

- AuthLayout
- AuthProvider
- LoginPage
- PersonalDetailsAuthForm
- RegisterPage
- ResendVerificationPage
- SignInCard2
- VerifyPage

Dashboard / Balance

- AccountStatement
- DashboardPage
- QuickContacts

Transfers

- EmptyRelationshipState
- RecipientStatusCard
- RelationshipSummaryCard
- TransferCheque
- TransferPage
- TransferQuoteSmallPrint
- UserProfileHeader
- UserProfilePage

Transactions / History

- RecentRelationshipTransactions
- TransactionDetailsDialog
- TransactionList
- TransactionReceipt
- TransactionsPage

AI Assistant

- AssistantBlocks
- ChatWidgetDemo
- FloatingChatWidget

Layout / Navigation

- App
- AppShell
- NotFoundPage
- NotFoundSlip
- RouteGuards
- ShellTopbar
- UserProfileSidebar

Shared UI

- AgentVideoSessionsPage (*fits none — video*)
- AnimatedText
- Avatar
- BentoCard
- Button (shadcn/ui)
- CurrencyProvider
- CurrencySelector
- JitsiMeeting (*fits none — video*)
- OrderConfirmationCard
- Primitives (UI toolkit)
- Select (shadcn/ui)
- SettingsPage (*fits none*)
- ShaderBackground
- VideoSessionPage (*fits none — video*)

Appendix A — Component index

Name	Path	Category	Area	Tier	Link
AuthLayout	client/src/ features/auth/ AuthLayout.ts x	layout	Auth	Lite	↗
AuthProvider	client/src/ features/auth/ AuthProvider. tsx	provider/context	Auth	Full	↗
LoginPage	client/src/ features/auth/ LoginPage.ts x	page	Auth	Full	↗
PersonalDetailsAuthForm	client/src/ features/ profile/ PersonalDetailsAuthForm.t sx	form	Auth	Full	↗
RegisterPage	client/src/ features/auth/ RegisterPage .tsx	page	Auth	Full	↗
ResendVerificationPage	client/src/ features/auth/ ResendVerificationPage.tsx	page	Auth	Full	↗
SignInCard2	client/src/ components/ ui/sign-in- card-2.tsx	form	Auth	Full	↗
VerifyPage	client/src/ features/auth/ VerifyPage.ts x	page	Auth	Full	↗
AccountStatement	client/src/ features/ dashboard/ AccountStatement.tsx	feature	Dashboard / Balance	Full	↗
		page		Full	↗

Name	Path	Category	Area	Tier	Link
DashboardPage	client/src/features/dashboard/DashboardPage.tsx		Dashboard / Balance		
QuickContacts	client/src/components/QuickContacts.tsx	feature	Dashboard / Balance	Full	↗
EmptyRelationshipState	client/src/features/users/EmptyRelationshipState.tsx	feature	Transfers	Full	↗
RecipientStatusCard	client/src/features/users/RecipientStatusCard.tsx	feature	Transfers	Full	↗
RelationshipSummaryCard	client/src/features/users/RelationshipSummaryCard.tsx	feature	Transfers	Full	↗
TransferCheque	client/src/components/TransferCheque.tsx	form	Transfers	Full	↗
TransferPage	client/src/features/transfer/TransferPage.tsx	page	Transfers	Full	↗
TransferQuoteSmallPrint	client/src/features/transfer/TransferQuoteSmallPrint.tsx	feature	Transfers	Full	↗
UserProfileHeader	client/src/features/	feature	Transfers	Full	↗

Name	Path	Category	Area	Tier	Link
	users/ UserProfileHeader.tsx				
UserProfilePage	client/src/ features/ users/ UserProfilePage.tsx	page	Transfers	Full	↗
RecentRelationshipTransactions	client/src/ features/ users/ RecentRelationshipTransactions.tsx	hook-bound container	Transactions / History	Full	↗
TransactionDetailsDialog	client/src/ components/ TransactionDetailsDialog.tsx	modal/overlay	Transactions / History	Full	↗
TransactionList	client/src/ components/ TransactionList.tsx	feature	Transactions / History	Full	↗
TransactionReceipt	client/src/ components/ TransactionReceipt.tsx	feature	Transactions / History	Full	↗
TransactionsPage	client/src/ features/ transactions/ TransactionsPage.tsx	page	Transactions / History	Full	↗
AssistantBlocks	client/src/ components/ assistant/ AssistantBlocks.tsx	feature	AI Assistant	Full	↗
ChatWidgetDemo	client/src/ components/ ui/floating- chat-widget- demo.tsx	page (demo harness)	AI Assistant	Full	↗

Name	Path	Category	Area	Tier	Link
FloatingChat Widget	client/src/components/ui/floating-chat-widget-shadcnui.tsx	hook-bound container	AI Assistant	Full	↗
App	client/src/app/App.tsx	layout (router)	Layout / Navigation	Full	↗
AppShell	client/src/components/AppShell.tsx	layout	Layout / Navigation	Full	↗
NotFoundPage	client/src/features/not-found/NotFoundPage.tsx	page	Layout / Navigation	Full	↗
NotFoundSlip	client/src/components/NotFoundSlip.tsx	feature	Layout / Navigation	Full	↗
RouteGuards	client/src/components/RouteGuards.tsx	provider/container	Layout / Navigation	Full	↗
ShellTopbar	client/src/components/ShellTopbar.tsx	layout	Layout / Navigation	Full	↗
UserProfileSidebar	client/src/components/ui/menu.tsx	layout	Layout / Navigation	Full	↗
AgentVideoSessionsPage	client/src/features/video/AgentVideoSessionsPage.tsx	page	Shared UI (fits none)	Full	↗
AnimatedText	client/src/components/ui/animated-text.tsx	primitive/ui	Shared UI	Lite	↗
Avatar		primitive/ui	Shared UI	Lite	↗

Name	Path	Category	Area	Tier	Link
	client/src/ components/ ui/avatar.tsx				
BentoCard	client/src/ components/ ui/bento- card.tsx	primitive/ui	Shared UI	Lite	↗
Button (shadcn/ui)	client/src/ components/ ui/button.tsx	primitive/ui	Shared UI	Lite	↗
CurrencyProv ider	client/src/ features/ currency/ CurrencyProv ider.tsx	provider/context	Shared UI	Full	↗
CurrencySele ctor	client/src/ features/ currency/ CurrencySele ctor.tsx	primitive/ui	Shared UI	Lite	↗
JitsiMeeting	client/src/ features/ video/ JitsiMeeting.t sx	feature	Shared UI (fits none)	Full	↗
OrderConfirm ationCard	client/src/ components/ ui/order- confirmation- card.tsx	primitive/ui	Shared UI	Lite	↗
Primitives (UI toolkit)	client/src/ components/ Primitives.tsx	primitive/ui	Shared UI	Lite	↗
Select (shadcn/ui)	client/src/ components/ ui/select.tsx	primitive/ui	Shared UI	Lite	↗
SettingsPage	client/src/ features/ settings/ SettingsPage .tsx	page	Shared UI (fits none)	Full	↗

Name	Path	Category	Area	Tier	Link
ShaderBackground	client/src/components/ui/shader-background.tsx	primitive/ui	Shared UI	Lite	↗
VideoSessionPage	client/src/features/video/VideoSessionPage.tsx	page	Shared UI (fits none)	Full	↗

Appendix B — Shared hooks, contexts, providers, and utilities

Non-visual modules the components depend on. They are excluded from the visual catalog but are essential context for it.

Contexts / providers

Module	Hook	Provides	Key endpoints
client/src/features/auth/AuthProvider.tsx	useAuth()	Session user , isAuthenticated , login / register / verify / logout , setSession , updateBalance .	GET /api/auth/me , POST /api/auth/{login,register,logout,resend-verification} , GET /api/auth/verify .
client/src/features/currency/CurrencyProvider.tsx	useCurrency()	Display currency , rates , conversionAvailable , formatAmount(amountInUs) .	GET /api/exchange-rates/current .

API client and types

Module	Role
client/src/lib/api.ts	The <code>api</code> object — every backend call (auth, accounts, transactions, transfer + quote, users, exchange rates, AI chat/stream/confirmation, video sessions). Adds CSRF headers on unsafe methods, parses SSE for streaming chat, and exposes <code>ApiError</code> (status + field <code>issues</code> + <code>details</code>) and

Module	Role
	<code>setUnauthorizedHandler</code> (401 → clear session).
<code>client/src/lib/types.ts</code>	All shared types: DTOs (<code>User</code> , <code>AccountSummary</code> , <code>Transaction</code> , <code>TransferQuote</code> , ...), the AI <code>AssistantResponseBlock</code> discriminated union, and <code>AiTransferConfirmation</code> (with <code>confirmAction</code> / <code>denyAction</code>).

Utilities

Module	Exports
<code>client/src/lib/format.ts</code>	<code>formatMoneyILS</code> , <code>formatCurrency</code> , <code>formatDate</code> , <code>formatRelativeDate</code> , <code>getInitials</code> .
<code>client/src/lib/currency.ts</code>	<code>SUPPORTED_DISPLAY_CURRENCIES</code> , <code>CURRENCY_LABELS</code> , <code>isDisplayCurrency</code> , <code>read/storeCurrency</code> , <code>convertILsForDisplay</code> , <code>formatILsAmount</code> .
<code>client/src/lib/validation.ts</code>	<code>validateEmail</code> , <code>validatePassword</code> , <code>validatePhone</code> , <code>validateAmount</code> , <code>validateReason</code> , <code>validateRequiredText</code> , <code>validateDateOfBirth</code> .
<code>client/src/lib/contacts.ts</code>	<code>getQuickContacts</code> — dedupes recent counterparties into { <code>email</code> , <code>avatar</code> } .
<code>client/src/lib/amount-words.ts</code>	<code>amountInWords</code> — cheque-style spelled amounts.
<code>client/src/lib/route-transition.ts</code>	<code>markAuthTransition</code> , <code>hasAuthTransition</code> , <code>clearAuthTransition</code> , <code>authTransitionState</code> — the auth → app entrance flag.
<code>client/src/lib/user-avatar.ts</code>	<code>getDisplayName</code> , <code>getInitial</code> , <code>createInitialAvatar</code> , <code>getUserAvatarUrl</code> .
<code>client/src/lib/utils.ts</code>	<code>cn</code> — <code>clsx</code> + <code>tailwind-merge</code> .
<code>client/src/main.tsx</code>	App entry: mounts <code>BrowserRouter</code> → <code>AuthProvider</code> → <code>App</code> .

Appendix C — Design tokens / theme

Virly runs **two coexisting token systems**:

1. Tailwind v4 `@theme inline ( client/src/index.css )`

shadcn-style HSL tokens consumed by `components/ui/*` and the AI assistant cards (via Tailwind utilities like `bg-primary`, `text-muted-foreground`).

Token	Value (HSL)	Approx.
<code>--background</code>	<code>40 33% 98%</code>	warm off-white
<code>--foreground</code>	<code>222 31% 12%</code>	near-black ink
<code>--card</code> / <code>--card-foreground</code>	<code>0 0% 100%</code> / <code>222 31% 12%</code>	white / ink
<code>--primary</code> / <code>--primary-foreground</code>	<code>168 76% 34%</code> / <code>0 0% 100%</code>	teal / white
<code>--destructive</code> / <code>--destructive-foreground</code>	<code>5 27% 42%</code> / <code>0 0% 100%</code>	muted red / white
<code>--muted</code> / <code>--muted-foreground</code>	<code>180 14% 91%</code> / <code>218 12% 45%</code>	pale teal / grey
<code>--border</code>	<code>180 14% 86%</code>	pale teal border
<code>--radius-xl</code>	<code>0.75rem</code>	—

2. Hand-written design system (`client/src/styles/global.css`)

Tokens used by the bespoke “paper/ledger” surfaces (`.cheque` , `.statement` , `.signin-*` , etc.).

Token	Value	Role
<code>--color-bg</code> / <code>--color-bg-soft</code>	<code>#e6eec9</code> / <code>#dce7bd</code>	page background
<code>--color-surface</code> / <code>--color-surface-raised</code>	translucent <code>#e6eec9</code>	cards / panels
<code>--color-text</code> / <code>--color-muted</code> / <code>--color-subtle</code>	<code>#173d42</code> / <code>rgba(23,61,66,.66)</code> / <code>rgba(23,61,66,.42)</code>	text
<code>--color-primary</code> / <code>--color-primary-deep</code> / <code>--color-primary-soft</code>	<code>#35858e</code> / <code>#1f5f67</code> / <code>#c2d099</code>	brand teal
<code>--color-credit</code> / <code>--color-debit</code>	<code>rgb(9,145,32)</code> / <code>rgba(190,9,9,.88)</code>	money in / out
	<code>#35858e</code> / <code>#7da78c</code> / <code>#7f5a55</code>	status

Token	Value	Role
<code>--color-success / --color-warning / --color-danger</code>		
<code>--font-body / --font-display</code>	"Hanken Grotesk" ... / "Fraunces" ...	typography
<code>--radius-card / --radius-control</code>	18px / 14px	corner radii
<code>--ease-spring</code>	<code>cubic-bezier(0.16, 1, 0.3, 1)</code>	shared motion easing
layout vars	<code>--page-max-width: 1180px , --page-gutter , --section-gap , --mobile-nav-offset</code>	layout rhythm

Note: `--color-border` is defined in **both** files with different values (Tailwind: `hsl(180 14% 86%)` ; global.css: `rgba(53,133,142,0.2)`); which one applies depends on whether an element is styled by Tailwind utilities or the hand-written classes.

Auth

The authentication area covers everything before a user reaches the application shell: the split-screen auth layout, the registration/login card, email verification, verification re-send, the post-auth “personal details” capture step, and the `AuthProvider` context that holds the session. Every network call here goes through `client/src/lib/api.ts`; the backend issues the session cookie and CSRF token, and the frontend never trusts a client-set identity. Screenshots below are placeholders (Storybook capture is in progress); filenames already follow the final convention so real images drop in with no edits.

Components in this area

- [AuthLayout](#)
- [AuthProvider](#)
- [LoginPage](#)
- [PersonalDetailsAuthForm](#)
- [RegisterPage](#)
- [ResendVerificationPage](#)
- [SignInCard2](#)
- [VerifyPage](#)

AuthLayout

- **Path:** `client/src/features/auth/AuthLayout.tsx`
- **Category:** layout | **Feature area:** Auth | **Tier:** Lite
- **Summary:** Split-screen auth shell with an animated brand/“visual” panel on one side and the form panel on the other, with a coordinated slide-out exit.

Screenshot(s)

Virly · Frontend component

AuthLayout

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Default: brand visual panel beside the form panel.

Virly · Frontend component

AuthLayout

state: exiting

PLACEHOLDER — replace via Storybook capture (1440x900)

`isExiting` — both panels blur and slide right as the app takes over.

Props / API

Prop	Type	Required	Default	Description
<code>title</code>	<code>string</code>	Yes	—	Heading shown when <code>barePanel</code> is false.
<code>subtitle</code>	<code>string</code>	Yes	—	Sub-heading; rendered only if non-empty.
<code>visualText</code>	<code>string</code>	No	—	If set, renders <code>AnimatedText</code> in the visual panel; otherwise a demo balance card.
<code>barePanel</code>	<code>boolean</code>	No	<code>false</code>	When true, renders <code>children</code> without the <code>.auth-card</code> wrapper.
<code>isExiting</code>	<code>boolean</code>	No	<code>false</code>	Drives the blur + slide-right exit animation on both panels.
<code>children</code>	<code>ReactNode</code>	Yes	—	Form content for the right panel.

States & variants

- `default` — both panels visible (`isExiting=false`).
- `exiting` — `isExiting=true` blurs/translates both panels (used during the hand-off to the app shell).
- Responsive: panels stack on narrow viewports (handled in `global.css`).

Usage example

```
<AuthLayout title="Sign in" subtitle="" visualText="Virly" barePanel isExiting={stage
=== "leaving"}>
  <SignInCard2 {...signInProps} />
</AuthLayout>
```

AuthProvider

- **Path:** `client/src/features/auth/AuthProvider.tsx`
- **Category:** provider/context | **Feature area:** Auth | **Tier:** Full
- **Summary:** React context that owns the authenticated `User` , exposes session actions (login/register/verify/logout), and bootstraps the session from `GET /api/auth/me` on mount.

Screenshot(s)

Virly · Frontend component

AuthProvider

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

No own UI; this placeholder represents the gated app once a session resolves.

Purpose & context

Wraps the entire app (mounted in `main.tsx` above `App`). On mount it calls `api.me()` and either sets or clears the session, which is what `RouteGuards` reads to decide between the auth pages and the app shell. It also registers an unauthorized handler so any `401` from the API client clears the session app-wide. It is the single client-side source of truth for “who is signed in” — but the backend remains authoritative; the provider only mirrors the server’s session.

Anatomy

- `AuthContext` (nullable) + `useAuth()` hook guard.
- Internal `AuthState` (`user` , `isLoading`).
- Memoized context value exposing `isAuthenticated` , `login` , `register` , `verify` , `resendVerification` , `logout` , `setSession` , `updateBalance` .

Props / API

Prop	Type	Required	Default	Description
<code>children</code>	<code>ReactNode</code>	Yes	—	The subtree that consumes auth context.

Exposed context value (`useAuth()`):

Member	Type	Description
<code>user</code>	<code>User null</code>	Current session user (mirrors server).
<code>isLoading</code>	<code>boolean</code>	True until the initial <code>me()</code> resolves.
<code>isAuthenticated</code>	<code>boolean</code>	<code>Boolean(user)</code> .
<code>login</code>	<code>(payload: LoginRequest) => Promise<User></code>	Calls <code>api.login</code> , sets session.
<code>register</code>	<code>(payload: RegisterRequest) => Promise<string></code>	Calls <code>api.register</code> ; returns server message.
<code>verify</code>	<code>(token: string) => Promise<User></code>	Calls <code>api.verify</code> , sets session.
<code>resendVerification</code>	<code>(email: string) => Promise<string></code>	Calls <code>api.resendVerification</code> .
<code>logout</code>	<code>() => Promise<void></code>	Calls <code>api.logout</code> (if signed in), then clears.
<code>setSession</code>	<code>(user: User) => void</code>	Replace the session user locally.
<code>updateBalance</code>	<code>(balance: number) => void</code>	Patch only the balance on the current user.

State & data

- Local state: `AuthState` via `useState`.
- Hooks: `useCallback`, `useEffect`, `useMemo`.
- Data: `api.me()` on mount; `api.login` / `api.register` / `api.verify` / `api.resendVerification` / `api.logout` via the exposed actions.
- Endpoints: `GET /api/auth/me`, `POST /api/auth/login`, `POST /api/auth/register`, `GET /api/auth/verify`, `POST /api/auth/resend-verification`, `POST /api/auth/logout`.

Interactions & events

- On mount → `me()` → `setSession` or `clearSession`.
- `setUnauthorizedHandler(clearSession)` so any `401` clears the session.
- `updateBalance` is called by transfer/AI flows after the server confirms a new balance — it never computes a balance itself.

States & variants

- `loading` — `isLoading=true` (guards render `null`).

- `authenticated` / `unauthenticated` — drive `RouteGuards`.
- `error/empty/success/disabled`: N/A (no own UI).

Dependencies

- Children: none rendered directly besides `AuthContext.Provider`.
- Libraries: React only. Uses `api`, `setUnauthorizedHandler` from `lib/api`.
- Styling: N/A.

Accessibility

N/A (no DOM). Consumers own their own a11y.

Usage example

```
// main.tsx
<AuthProvider>
  <App />
</AuthProvider>

// any component
const { user, isAuthenticated, logout } = useAuth();
```

Related / used by

Mounted by `main.tsx`. Consumed by `RouteGuards`, `AppShell`, `ShellTopbar`, `DashboardPage`, `TransferPage`, `SettingsPage`, `FloatingChatWidget`, and the auth pages.

Notes / gotchas

`useAuth()` throws if used outside the provider. `updateBalance` is a no-op if there is no current user.

LoginPage

- **Path:** `client/src/features/auth/LoginPage.tsx`
- **Category:** page | **Feature area:** Auth | **Tier:** Full
- **Summary:** Sign-in page that orchestrates the `SignInCard2` form, the post-login “personal details” step, and the animated hand-off into the app.

Screenshot(s)

Virly · Frontend component

LoginPage

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Default sign-in card inside the auth layout.

Virly · Frontend component

LoginPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Invalid credentials / form error surfaced on the card.

Virly · Frontend component

LoginPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Submitting — spinner on the submit button, exit animation arming.

Virly · Frontend component

LoginPage

state: personal-details

PLACEHOLDER — replace via Storybook capture (1440x900)

`needsPersonalDetails` users see `PersonalDetailsAuthForm` before the dashboard.

Purpose & context

The entry point of the protected app for returning users. It validates email + password locally, calls `auth.login`, and on success either advances to the personal-details capture (if the server flags `needsPersonalDetails`) or navigates to the originally-requested route (or `/dashboard`) with the `auth-transition` flag set so the shell plays its entrance animation.

Anatomy

- `AuthLayout` wrapper (bare panel, `visualText="Virly"`).
- `AnimatePresence` switching between `login` and `personalDetails` stages.
- `SignInCard2` (controlled) for the credential form.
- `PersonalDetailsAuthForm` for the second stage.

Props / API

None. (Reads route `location.state.from` for post-login redirect.)

State & data

- Local state: `email`, `password`, `rememberMe`, `errors`, `isSubmitting`, `stage` (`"login"` | `"personalDetails"` | `"leaving"`).
- Hooks: `useAuth`, `useNavigate`, `useLocation`, `useState`.
- Data: `auth.login` → `POST /api/auth/login`.

Interactions & events

- Submit → `validateEmail` / `validatePassword` → `auth.login`.
- 403 (unverified) → redirects to `/resend-verification` with the email in route state.
- Other errors → `errors.form`.
- Success with `needsPersonalDetails` → stage `personalDetails`; otherwise `markAuthTransition()` + navigate to redirect target.

States & variants

- `default`, `loading` (`isSubmitting`), `error` (field/form), `personalDetails` stage, `leaving` (exit animation). Empty/disabled: N/A.

Dependencies

- Children: `AuthLayout`, `SignInCard2`, `PersonalDetailsAuthForm`.
- Libraries: `framer-motion`, `react-router-dom`.
- Helpers: `validateEmail`, `validatePassword`, `markAuthTransition`, `clearAuthTransition`, `authTransitionState`.

Accessibility

Form semantics and errors come from `SignInCard2` (`role="alert"` on form error). Stage transitions use `AnimatePresence`; `MotionConfig reducedMotion` in `App` respects reduced-motion. TODO: confirm focus moves to the personal-details form on stage change.

Usage example

```
<Route path="/login" element={<GuestRoute><LoginPage /></GuestRoute>} />
```

Related / used by

Routed by `App` inside `GuestRoute`. Pairs with `RegisterPage`, `ResendVerificationPage`, `VerifyPage`.

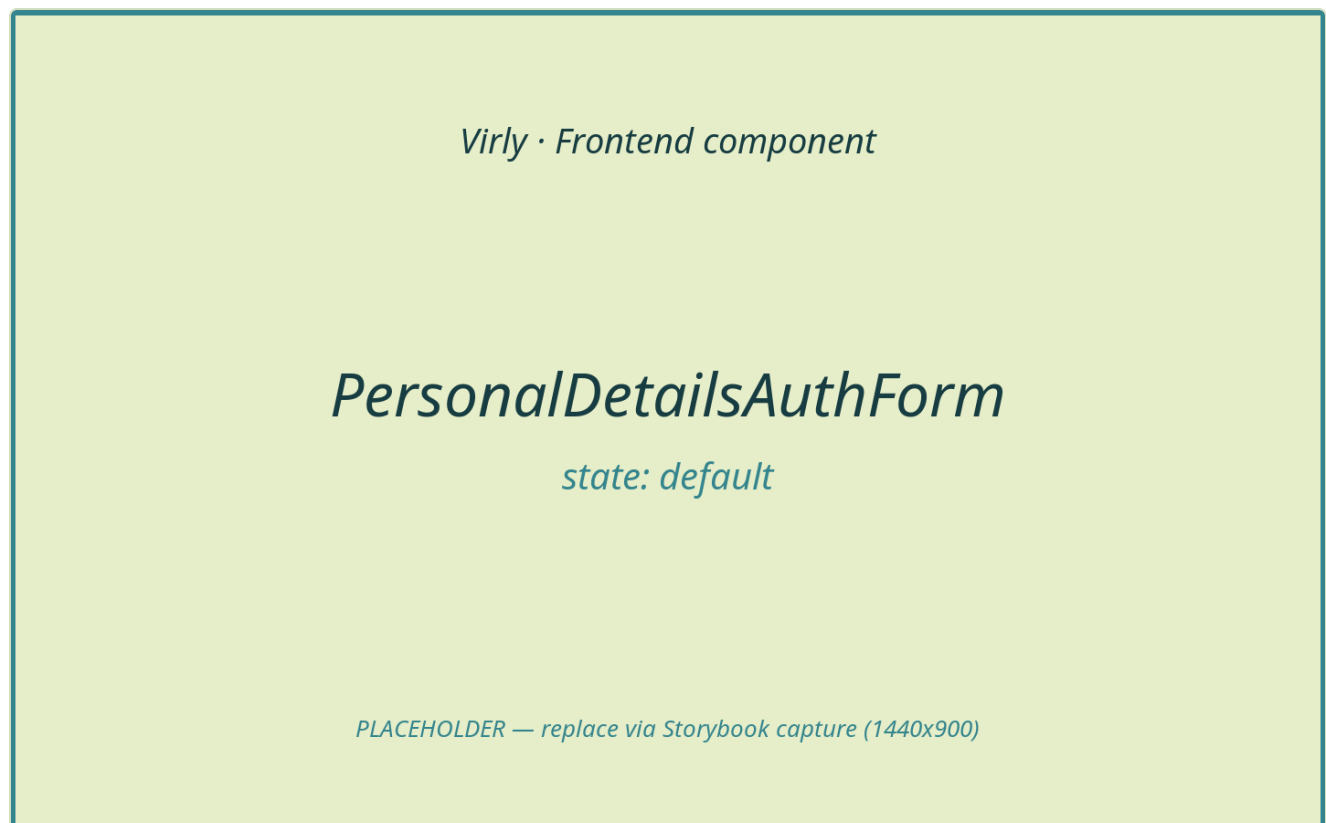
Notes / gotchas

The `leaving` stage waits `authExitMs` (1000ms) in parallel with the login request so the visual exit and the network call finish together.

PersonalDetailsAuthForm

- **Path:** `client/src/features/profile/PersonalDetailsAuthForm.tsx`
- **Category:** form | **Feature area:** Auth | **Tier:** Full
- **Summary:** Post-authentication KYC-style form that captures name, date of birth, and address, with a “Skip for now” escape hatch.

Screenshot(s)



Empty personal-details form (animated card).

Virly · Frontend component

PersonalDetailsAuthForm

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Per-field and form-level validation errors.

Virly · Frontend component

PersonalDetailsAuthForm

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Submitting — spinner in the primary button.

Purpose & context

Shown by `LoginPage` and `VerifyPage` when the server marks a user `needsPersonalDetails`. It collects the customer profile, persists it via `api.updatePersonalDetails`, patches the local session, then calls `onComplete` to continue into the app. The user may skip (`api.skipPersonalDetails`).

Anatomy

- Animated `signin-card` shell with logo + “Personal details” header.
- Two-column grids of icon-prefixed fields (`renderField`).
- Fields: `firstName`, `lastName`, `dateOfBirth`, `country`, `stateRegion`, `city`, `postalCode`, `street`, `addressLine2`.
- Primary “Save details” submit + secondary “Skip for now”.

Props / API

Prop	Type	Required	Default	Description
<code>onComplete</code>	<code>() => void</code>	Yes	—	Called after a successful save or skip.

State & data

- Local state: `form` (`FormState`), `errors` (`FormErrors`), `focusedField`, `isSubmitting`, `isSkipping`.
- Hooks: `useAuth`, `useState`.
- Data: `api.updatePersonalDetails` → `PUT /api/accounts/personal-details`; `api.skipPersonalDetails` → `POST /api/accounts/personal-details/skip`.

Interactions & events

- Submit → `validateForm` (required text, DOB) → `updatePersonalDetails` → `auth.setSession({ ...needsPersonalDetails: false })` → `onComplete`.
- Skip → `skipPersonalDetails` → `onComplete`.
- Server field issues map back onto inputs (e.g. `address.country`).

States & variants

- `default`, `error` (field + `errors.form`), `loading` (`isSubmitting`), `skipping` (`isSkipping`). Empty/success/disabled handled inline (buttons disabled while busy).

Dependencies

- Libraries: `framer-motion`, `lucide-react` icons.
- Helpers: `validateRequiredText`, `validateDateOfBirth`.
- Styling: `signin-*` / `profile-auth-*` classes (`global.css`).

Accessibility

Inputs are labelled via `htmlFor / id`; `aria-invalid` reflects errors; form-level error uses `role="alert"`. TODO: associate field errors with inputs via `aria-describedby`.

Usage example

```
<PersonalDetailsAuthForm onComplete={finishAuthFlow} />
```

Related / used by

Rendered by `LoginPage` and `VerifyPage`. Mirrors the editable details form in `SettingsPage` (same validators, same endpoint).

Notes / gotchas

`stateRegion` and `addressLine2` are optional and are not validated client-side.

RegisterPage

- **Path:** `client/src/features/auth/RegisterPage.tsx`
- **Category:** page | **Feature area:** Auth | **Tier:** Full
- **Summary:** Account-creation page driving the `SignInCard2` register variant (email, password, confirm password, phone).

Screenshot(s)



Registration card with email/password/confirm/phone.

Virly · Frontend component

RegisterPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Validation and duplicate-email (409) errors.

Virly · Frontend component

RegisterPage

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Success message prompting the user to verify their email.

Virly · Frontend component

RegisterPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Submitting — spinner in the submit button.

Purpose & context

Lets a new user create an account. Validates email, password (min 8), matching confirmation, and phone, then calls `auth.register`. On success it shows the server message (a “check your email to verify” prompt); the user does not log in until they verify.

Anatomy

`AuthLayout` (bare panel) wrapping a single `SignInCard2` configured for registration (confirm-password + phone fields, “Create account” labels).

Props / API

None.

State & data

- Local state: `email`, `password`, `confirmPassword`, `phone`, `message`, `errors`, `isSubmitting`.
- Hooks: `useAuth`, `useState`.
- Data: `auth.register` → `POST /api/auth/register`.

Interactions & events

- Submit → `validateEmail` / `validatePassword("register")` / confirm match / `validatePhone` → `auth.register` → `setMessage`.
- 409 → duplicate-email message on the email field.

- Other `ApiError` issues map onto fields.

States & variants

- `default`, `loading`, `error`, `success` (message banner). Empty/disabled: N/A.

Dependencies

- Children: `AuthLayout`, `SignInCard2`.
- Helpers: `validateEmail`, `validatePassword`, `validatePhone`.

Accessibility

Inherits `SignInCard2` semantics (labelled inputs, password show/hide button, `role="status"` success, `role="alert"` form error).

Usage example

```
<Route path="/register" element={<GuestRoute><RegisterPage /></GuestRoute>} />
```

Related / used by

Routed by `App` inside `GuestRoute`. Links to `/login` via the card footer.

Notes / gotchas

Passwords are sent over the API client (HTTPS in production) and never stored client-side; the backend hashes them. The frontend must never persist or log the password.

ResendVerificationPage

- **Path:** `client/src/features/auth/ResendVerificationPage.tsx`
- **Category:** page | **Feature area:** Auth | **Tier:** Full
- **Summary:** Lets an unverified user request a fresh verification email, seeded from the email passed in route state.

Screenshot(s)

Virly · Frontend component

ResendVerificationPage

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Single email field + "Send verification link".

Virly · Frontend component

ResendVerificationPage

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Confirmation that the link was sent.

Virly · Frontend component

ResendVerificationPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Invalid email or server error.

Purpose & context

Reached when login returns `403` (account not verified), or directly from the verify-failure screen. Pre-fills the email from `location.state.email` and posts to the resend endpoint.

Anatomy

`AuthLayout` wrapping a `form-stack`: success/error banners, a single `Field`, a submit `Button`, and a “Sign in” link.

Props / API

None.

State & data

- Local state: `email` (seeded), `error`, `message`, `isSubmitting`.
- Hooks: `useAuth`, `useLocation`, `useState`.
- Data: `auth.resendVerification` → `POST /api/auth/resend-verification`.

Interactions & events

Submit → `validateEmail` → `auth.resendVerification` → success message (or error string).

States & variants

- `default`, `loading` (button label “Sending...”), `success`, `error`. Empty/disabled: N/A.

Dependencies

- Children: `AuthLayout`, `Field`, `Button`, `ErrorBanner`, `SuccessBanner`.
- Helpers: `validateEmail`.

Accessibility

`Field` is labelled; banners carry `role="alert"` / `role="status"`.

Usage example

```
<Route path="/resend-verification" element={<ResendVerificationPage />} />
```

Related / used by

Linked from `LoginPage` (403 flow) and `VerifyPage` (failure). Not wrapped in `GuestRoute` — reachable while logged out.

Notes / gotchas

Always shows the server's message verbatim; the endpoint is intentionally non-committal about whether the address exists.

SignInCard2

- **Path:** `client/src/components/ui/sign-in-card-2.tsx`
- **Category:** form | **Feature area:** Auth | **Tier:** Full
- **Summary:** The reusable, heavily-animated credential card used for both login and registration; fully controlled by its parent page.

Screenshot(s)

Virly · Frontend component

SignInCard2

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Default login configuration (email, password, remember-me).

Virly · Frontend component

SignInCard2

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Field + form errors rendered under inputs / above the form.

Virly · Frontend component

SignInCard2

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Register variant showing the success message banner.

Virly · Frontend component

SignInCard2

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Submit button showing the spinner while `isLoading`.

Purpose & context

A presentational, controlled form card with a 3D tilt-on-hover effect, animated light beams, and password visibility toggle. The same component serves login (`onSubmit` , remember-me) and registration (extra `confirmPassword` + `phone` fields appear when their `onChange` handlers are provided). All state lives in the parent (`LoginPage` / `RegisterPage`).

Anatomy

- Tilt wrapper (`useMotionValue` / `useTransform` from pointer position).
- Glow + light-frame beams (decorative).
- Header: logo “V” + title.
- Form: optional form-error + success banners; email; password (+ show/hide); optional confirm-password; optional phone; optional remember-me; animated submit; footer link.

Props / API

Prop	Type	Required	Default	Description
<code>title</code>	<code>string</code>	No	<code>"Welcome Back"</code>	Card heading.
<code>submitLabel</code>	<code>string</code>	No	<code>"Sign In"</code>	Submit button text.
<code>footerLabel</code>	<code>string</code>	No	<code>"Create account"</code>	Footer link text.
<code>footerTo</code>	<code>string</code>	No	<code>"/register"</code>	Footer link target.
<code>email</code>	<code>string</code>	Yes	—	Controlled email value.
<code>password</code>	<code>string</code>	Yes	—	Controlled password value.
<code>rememberMe</code>	<code>boolean</code>	No	<code>false</code>	Remember-me checkbox state.
<code>showRememberMe</code>	<code>boolean</code>	No	<code>true</code>	Show the remember-me row (hidden when <code>onPhoneChange</code> is set).
<code>confirmPassword</code>	<code>string</code>	No	—	Controlled confirm value (register).
<code>phone</code>	<code>string</code>	No	—	Controlled phone value (register).
<code>emailError</code> / <code>passwordError</code> / <code>confirmPasswordError</code> /	<code>string</code>	No	—	Per-field error text.

Prop	Type	Required	Default	Description
<code>phoneError</code>				
<code>formError</code>	<code>string</code>	No	—	Form-level error banner.
<code>successMessage</code>	<code>string</code>	No	—	Success banner text.
<code>isLoading</code>	<code>boolean</code>	Yes	—	Disables submit and shows the spinner.
<code>onEmailChange</code> / <code>onPasswordChange</code>	<code>(value: string) => void</code>	Yes	—	Field change handlers.
<code>onRememberMeChange</code>	<code>(value: boolean) => void</code>	No	—	Remember-me handler.
<code>onConfirmPasswordChange</code>	<code>(value: string) => void</code>	No	—	Presence enables the confirm field (register mode).
<code>onPhoneChange</code>	<code>(value: string) => void</code>	No	—	Presence enables the phone field.
<code>onSubmit</code>	<code>(event: FormEvent<HTMLFormElement>) => void</code>	Yes	—	Form submit handler.

State & data

- Local state: `showPassword` , `focusedInput` , motion values for tilt.
- No data fetching — purely controlled.

Interactions & events

- `onSubmit` bubbles to the parent (which calls the API).
- Show/hide password toggles `type` between `text` / `password` .
- Tilt follows the pointer but ignores movement over inputs/buttons/links.

States & variants

- `default` (login), register variant (confirm + phone), `loading` , `error` (field + form), `success` . Disabled: submit disabled while `isLoading` .

Dependencies

- Libraries: `framer-motion`, `lucide-react`, `react-router-dom` (footer link).
- Helpers: `cn`. Styling: `signin-*` classes (`global.css`).

Accessibility

Inputs are labelled (icon `aria-hidden`, real `htmlFor / id`); `aria-invalid` on errored inputs; password toggle has `aria-label`; form error `role="alert"`, success `role="status"`. TODO: link field errors to inputs via `aria-describedby`.

Usage example

```
<SignInCard2
  email={email}
  password={password}
  isLoading={isSubmitting}
  emailError={errors.email}
  onEmailChange={setEmail}
  onPasswordChange={setPassword}
  onSubmit={handleSubmit}
/>
```

Related / used by

Rendered by `LoginPage` and `RegisterPage`.

Notes / gotchas

Whether the card is in “register” mode is inferred from the presence of `onConfirmPasswordChange` / `onPhoneChange` rather than an explicit flag.

VerifyPage

- **Path:** `client/src/features/auth/VerifyPage.tsx`
- **Category:** page | **Feature area:** Auth | **Tier:** Full
- **Summary:** Consumes the `?token=` link from a verification email, verifies it, and routes to personal-details or the dashboard.

Screenshot(s)

Virly · Frontend component

VerifyPage

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Checking token (spinner panel).

Virly · Frontend component

VerifyPage

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Verified — opening the dashboard.

Virly · Frontend component

VerifyPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Missing/invalid token with a “Resend verification” CTA.

Purpose & context

Landing page for the email verification link. On mount it reads the `token` query param and calls `auth.verify`. Success either advances to the personal-details step (`needsPersonalDetails`) or briefly shows a success banner and navigates to `/dashboard` with the auth-transition flag.

Anatomy

`AuthLayout` wrapping an `AnimatePresence` of two stages — the verify status panel (spinner / success / error + resend link) and `PersonalDetailsAuthForm`.

Props / API

None. (Reads `token` from `useSearchParams`.)

State & data

- Local state: `error`, `done`, `stage` (`"verify"` | `"personalDetails"` | `"leaving"`).
- Hooks: `useAuth`, `useNavigate`, `useSearchParams`, `useEffect`, `useState`.
- Data: `auth.verify` → `GET /api/auth/verify?token=...`.

Interactions & events

- Mount with token → `verify`. No token → “token is missing” error.
- Success → either personal-details stage or success → navigate `/dashboard`.
- Failure → error + “Resend verification” link.

States & variants

- `default` (`checking`), `success` (`done`), `error`, `personalDetails` `stage`, `leaving`. `Empty/` disabled: N/A.

Dependencies

- Children: `AuthLayout`, `PersonalDetailsAuthForm`, `ErrorBanner`, `SuccessBanner`.
- Libraries: `framer-motion`, `react-router-dom`.

Accessibility

Banners carry `role` semantics; the checking state is a visible text panel. Not wrapped in `GuestRoute` (verification can happen in any session state).

Usage example

```
<Route path="/verify" element={<VerifyPage />} />
```

Related / used by

Routed by `App`. Hands off to `PersonalDetailsAuthForm`; links to `ResendVerificationPage` on failure.

Notes / gotchas

The verify effect guards against double-invocation with an `active` flag and ignores results after unmount.

Dashboard / Balance

The dashboard is the authenticated landing surface. It greets the user, fetches the account summary (balance + recent ledger), and renders Virly's signature "bank statement" alongside quick-send shortcuts and activity totals. The displayed balance is always the server's figure: the page calls `api.accountSummary` and pushes the returned balance into `AuthProvider` via `updateBalance` — it never derives a balance from the transaction list. Screenshots are placeholders pending Storybook capture.

Components in this area

- [AccountStatement](#)
 - [DashboardPage](#)
 - [QuickContacts](#)
-

AccountStatement

- **Path:** `client/src/features/dashboard/AccountStatement.tsx`
- **Category:** feature | **Feature area:** Dashboard / Balance | **Tier:** Full
- **Summary:** A printed-statement rendering of the account: masthead, closing balance, money-in/out totals, and a clickable ledger of recent transactions.

Screenshot(s)

Virly · Frontend component

AccountStatement

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Default statement with ledger rows and a closing balance.

Virly · Frontend component

AccountStatement

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

No transactions — `EmptyState` with a “Make a transfer” CTA.

Purpose & context

Renders the main dashboard column as a paper bank statement. It receives an already-fetched `AccountSummary` and computes a running balance by walking the closing balance backwards through the (newest-first) entries. Each ledger row is a button that opens the shared `TransactionDetailsDialog`.

Anatomy

- Masthead: brand mark, “Account Statement”, holder, masked account number, period.
- Summary band: closing balance + “as of” date; brought-forward, money-in, money-out figures.
- Ledger: header row + a button per transaction (date, counterparty + reason, paid-out/paid-in amount, running balance).
- Footer: entry count + “View all transactions” link.
- Empty branch: `EmptyState` with a transfer CTA.

Props / API

Prop	Type	Required	Default	Description
<code>summary</code>	<code>AccountSummary</code>	Yes	—	Balance + transactions + pagination.
<code>holderName</code>	<code>string</code>	Yes	—	Display name in the masthead.
<code>accountNumber</code>	<code>string</code>	Yes	—	Decorative masked account number.
<code>formatAmount</code>	<code>(amount: number) => string</code>	Yes	—	Display-currency formatter (from <code>useCurrency</code>).
<code>onSelectTransaction</code>	<code>(transaction: Transaction) => void</code>	Yes	—	Opens the details dialog.

State & data

- No local state; derives totals, `balanceAfter[]`, `broughtForward`, and the statement period from `summary.transactions`.
- No data fetching of its own (parent fetches `api.accountSummary`).

Interactions & events

- Clicking a ledger row → `onSelectTransaction(line)`.
- “View all transactions” → `/transactions`.

States & variants

- `default` (has entries), `empty` (no entries). Loading is owned by the parent (`Skeleton`). Error/success/disabled: N/A.

Dependencies

- Children: `EmptyState` , `Link` .
- Styling: `statement-*` classes (`global.css`); `Intl.DateTimeFormat` .

Accessibility

`aria-label="Account statement"` on the section; each ledger row is a `button` with a descriptive `aria-label` (date, counterparty, direction, amount, balance). Decorative flourishes are `aria-hidden` .

Usage example

```
<AccountStatement
  summary={summary}
  holderName={greetingName}
  accountNumber={accountNumber}
  formatAmount={formatAmount}
  onSelectTransaction={setSelectedTransaction}
/>
```

Related / used by

Rendered by `DashboardPage` . Opens `TransactionDetailsDialog` .

Notes / gotchas

The running balance is a client-side reconstruction for display; the closing balance (`summary.balance`) is the authoritative server figure.

DashboardPage

- **Path:** `client/src/features/dashboard/DashboardPage.tsx`
- **Category:** page | **Feature area:** Dashboard / Balance | **Tier:** Full
- **Summary:** The authenticated home page: greeting, account statement, quick send, and activity stats, with an entrance animation after sign-in.

Screenshot(s)

Virly · Frontend component

DashboardPage

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Loaded dashboard: statement + Quick Send + Activity Stats.

Virly · Frontend component

DashboardPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Initial load — `Skeleton` placeholder.

Virly · Frontend component

DashboardPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Account load failed — `ErrorBanner`.

Purpose & context

The default route after login. On mount it fetches the account summary, mirrors the balance into the auth context, and lays out the statement (main column) beside Quick Send and Activity Stats (side column). When the user arrived from the auth flow it plays a staggered entrance animation.

Anatomy

- `PageHeader` greeting (“Hello, {name}”) + “Transfer” CTA.
- Main column: `AccountStatement`.
- Side column: Quick Send card (`QuickContacts`) + Activity Stats card (received/sent totals).
- `TransactionDetailsDialog` for the selected ledger row.

Props / API

None.

State & data

- Local state: `summary`, `selectedTransaction`, `error`, `isLoading`, `enteredFromAuth`.
- Hooks: `useAuth`, `useCurrency`, `useLocation`, `useNavigate`, `useMemo`.
- Data: `api.accountSummary(1, 10)` → `GET /api/accounts/me?page=1&limit=10`; pushes balance to `auth.updateBalance`.

Interactions & events

- Mount → fetch summary → `setSummary` + `auth.updateBalance`.
- Quick Send select → stores `virly-prefill-recipient` in `sessionStorage` and navigates to `/transfer`.
- Ledger row → `setSelectedTransaction` (opens dialog).

States & variants

- `default`, `loading` (`Skeleton`), `error` (`ErrorBanner`). The statement's own `empty` state shows when there are no transactions. Success/disabled: N/A.

Dependencies

- Children: `PageHeader`, `Card`, `ErrorBanner`, `Skeleton`, `QuickContacts`, `AccountStatement`, `TransactionDetailsDialog`.
- Libraries: `framer-motion`, `lucide-react`, `react-router-dom`.
- Helpers: `getQuickContacts`, `hasAuthTransition`, `clearAuthTransition`.

Accessibility

Section headings (`<h2>`) structure the side panels; activity direction marks are `aria-hidden`. Entrance animation respects `MotionConfig reducedMotion`.

Usage example

```
<Route path="/dashboard" element={<DashboardPage />} />
```

Related / used by

Routed inside the protected shell. Feeds `AccountStatement`, `QuickContacts`, and the shared `TransactionDetailsDialog`.

Notes / gotchas

Received/sent totals are computed from the first page of transactions only (the summary endpoint returns 10), so the “Activity Stats” numbers are a recent snapshot, not lifetime totals.

QuickContacts

- **Path:** `client/src/components/QuickContacts.tsx`
- **Category:** feature | **Feature area:** Dashboard / Balance | **Tier:** Full
- **Summary:** A “Quick Send” list of recent counterparties; each row selects a recipient and links to that user’s profile.

Screenshot(s)

Virly · Frontend component

QuickContacts

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

List of recent contacts with initials avatars.

Virly · Frontend component

QuickContacts

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

“No contacts” placeholder when the list is empty.

Purpose & context

Renders the dashboard's Quick Send shortcuts from deduplicated recent counterparties (`getQuickContacts`). Selecting a contact hands the recipient to the transfer flow (the parent stores the prefill and navigates); a secondary link opens the contact's profile.

Anatomy

- Empty branch: `quick-contact-empty`.
- Per contact: a select button (avatar initials + email) + a profile link (`/users/:email`).

Props / API

Prop	Type	Required	Default	Description
<code>contacts</code>	<code>QuickContact[]</code>	Yes	—	<code>{ email, avatar }</code> items.
<code>onSelectContact</code>	<code>(email: string) => void</code>	Yes	—	Fired when a contact is chosen.

State & data

None (presentational). Contacts are derived upstream from the account summary.

Interactions & events

- Contact button → `onSelectContact(email)` (dashboard prefills + navigates to `/transfer`).
- Profile link → `/users/:email`.

States & variants

- `default` (has contacts), `empty`. Loading/error/success/disabled: N/A.

Dependencies

- Children: `Link` (`react-router-dom`), `LucideReact` `UserRound`.
- Helper type: `QuickContact` from `lib/contacts`.

Accessibility

Profile links carry `aria-label / title` ("View ...'s profile"); avatars are `aria-hidden`.

Usage example

```
<QuickContacts
  contacts={quickContacts}
  onSelectContact={(email) => {
    sessionStorage.setItem("virly-prefill-recipient", email);
    navigate("/transfer");
  }}
/>
```

Related / used by

Rendered by `DashboardPage` . The selection handoff feeds `TransferPage` (which reads `virly-prefill-recipient`).

Notes / gotchas

`onSelectContact` only **prefills** a recipient — it does not initiate or confirm a transfer. The user still completes amount entry, review, and the explicit confirmation on `TransferPage` .

Transfers

The transfers area is where money is prepared and sent. Its centrepiece is the cheque metaphor: the user “writes a cheque” (form), “reviews” it (with an authoritative server quote for non-ILS amounts), then “signs & sends”. This area also includes the user-profile / recipient cluster (`features/users/*`), which is the recipient-context surface — verification status, relationship history, and a “Transfer” CTA that prefills the recipient into the same cheque flow.

Architecture, in one line: the UI only *prepares and initiates* a transfer. The Node/Express + MongoDB backend is authoritative for balances and is the only thing that moves money. A transfer executes solely when the user confirms and the client calls the execution endpoint (`POST /api/transactions`); the recipient cards and profile page never debit a balance — they only seed the form. Every component below carries the **Architecture constraints** callout.

Screenshots are placeholders pending Storybook capture; filenames follow the final convention.

Components in this area

- `EmptyRelationshipState`
- `RecipientStatusCard`
- `RelationshipSummaryCard`
- `TransferCheque`
- `TransferPage`
- `TransferQuoteSmallPrint`
- `UserProfileHeader`
- `UserProfilePage`

EmptyRelationshipState

- **Path:** `client/src/features/users/EmptyRelationshipState.tsx`
- **Category:** feature | **Feature area:** Transfers | **Tier:** Full
- **Summary:** Empty-state card shown on a profile when you and the viewed user have no shared transactions, with an optional “Transfer” CTA.

Screenshot(s)

Virly · Frontend component

EmptyRelationshipState

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

No shared history, with a Transfer CTA when transfers are allowed.

Virly · Frontend component

EmptyRelationshipState

state: disabled

PLACEHOLDER — replace via Storybook capture (1440x900)

Same card without the CTA when `canSendMoney` is false.

Architecture constraints

- Backend is authoritative; this component does not mutate balances client-side.
- The “Transfer” CTA only prefills the recipient and opens the transfer flow; the explicit confirmation + `POST /api/transactions` still gate execution.
- AI Assistant: N/A.

Purpose & context

Rendered by `UserProfilePage` when `relationship.transactionCount === 0`. It invites the viewer to start a relationship by sending a first transfer.

Anatomy

A `card` wrapping `EmptyState` (HandCoins icon, title/message) with an optional `Button` (“Transfer”).

Props / API

Prop	Type	Required	Default	Description
<code>viewedName</code>	<code>string</code>	Yes	—	Display name of the viewed user.
<code>canSendMoney</code>	<code>boolean</code>	Yes	—	Whether to render the Transfer CTA.
<code>onSendMoney</code>	<code>() => void</code>	Yes	—	Prefills recipient + navigates to transfer.

State & data

None (presentational).

Interactions & events

“Transfer” button → `onSendMoney` (parent prefills + navigates).

States & variants

- `default` (CTA present), `disabled` (no CTA when `canSendMoney` is false). Loading/empty/error/success: N/A.

Dependencies

- Children: `EmptyState`, `Button`, `lucide-react` `HandCoins`.

Accessibility

`aria-label="No shared history"` on the section; `EmptyState` provides heading + message.

Usage example

```
<EmptyRelationshipState  
  viewedName={user.displayName}  
  canSendMoney={relationship.canTransferToUser}  
  onSendMoney={handleSendMoney}  
>  
</>
```

Related / used by

Rendered by `UserProfilePage` . Sibling of `RelationshipSummaryCard` .

Notes / gotchas

`canSendMoney` derives from the server's `relationship.canTransferToUser` ; the client trusts the server's eligibility decision.

RecipientStatusCard

- **Path:** `client/src/features/users/RecipientStatusCard.tsx`
- **Category:** feature | **Feature area:** Transfers | **Tier:** Full
- **Summary:** Sidebar card summarising whether the viewed user is a verified recipient and whether you can transfer to them.

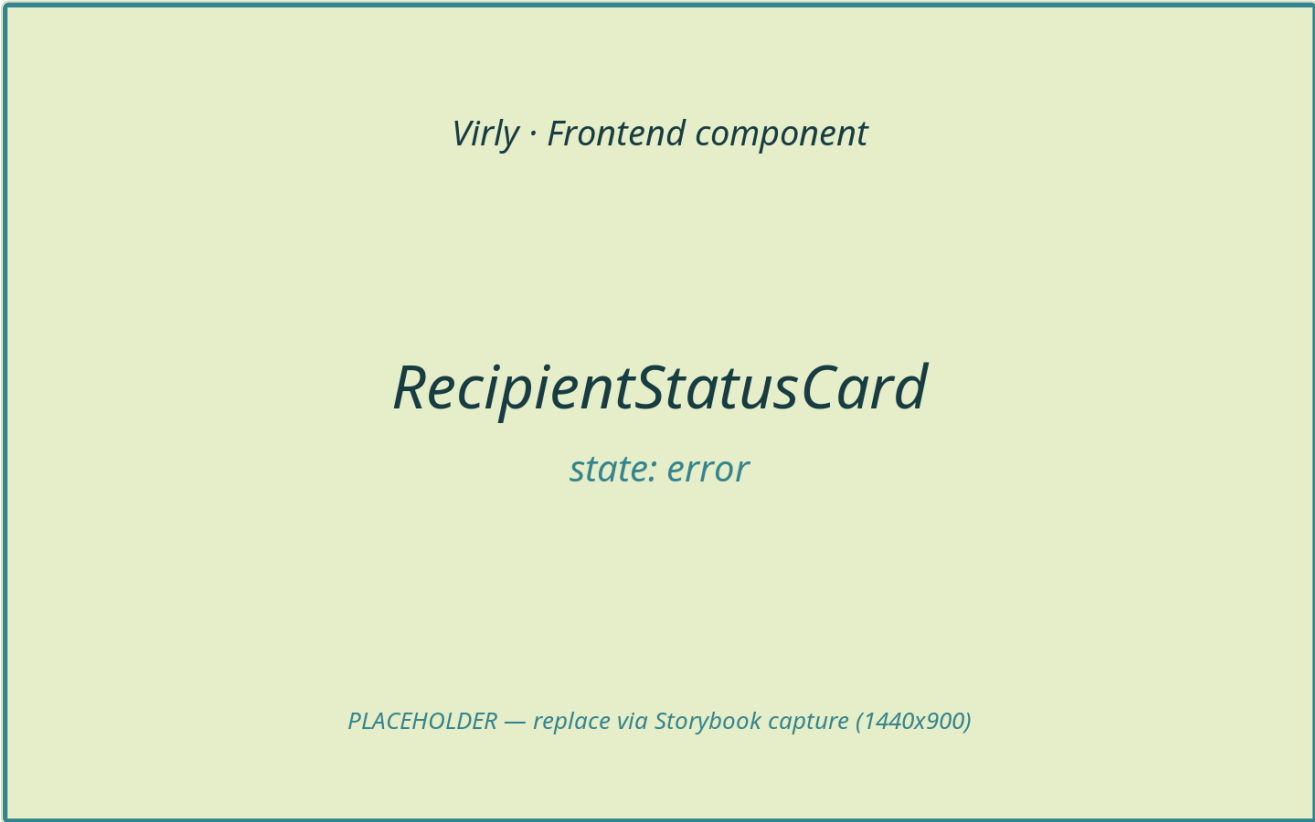
Screenshot(s)

Virly · Frontend component

RecipientStatusCard

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)



“Not verified yet” — transfers allowed but flagged for extra care.

Architecture constraints

- Backend is authoritative; verification + transfer eligibility come from the server (`relationship.isVerifiedRecipient` , `canTransferToUser`).
- The “Transfer” CTA only prefills the recipient; execution still requires the explicit confirmation and `POST /api/transactions` .
- AI Assistant: N/A.

Purpose & context

Rendered in the profile sidebar for users other than yourself. It maps the relationship status to a verified / not-verified / self message and offers a Transfer CTA when allowed.

Anatomy

Status head (icon + title from `getStatusCopy`), message paragraph, optional `Button` (“Transfer”).

Props / API

Prop	Type	Required	Default	Description
		Yes	—	Server relationship summary.

Prop	Type	Required	Default	Description
<code>relationship</code>	<code>UserRelationshipSummary</code>			
<code>viewedName</code>	<code>string</code>	Yes	—	Display name of the viewed user.
<code>onSendMoney</code>	<code>() => void</code>	Yes	—	Prefills recipient + navigates to transfer.

State & data

None; copy is derived from `relationship.relationshipStatus` / `isVerifiedRecipient`.

Interactions & events

“Transfer” button (when `canTransferToUser`) → `onSendMoney`.

States & variants

- `default` (verified recipient), `error`-toned “not verified yet” warning, `self` (own account). CTA hidden when `canTransferToUser` is false.

Dependencies

- Children: `Button`, `lucide-react` (`BadgeCheck`, `CircleHelp`, `Send`, `UserCircle2`).

Accessibility

`aria-label="Recipient status"`; icons `aria-hidden`. The verified state gets a distinct icon style.

Usage example

```
<RecipientStatusCard
  relationship={relationship}
  viewedName={user.displayName}
  onSendMoney={handleSendMoney}
/>
```

Related / used by

Rendered by `UserProfilePage` (sidebar). Complements `RelationshipSummaryCard`.

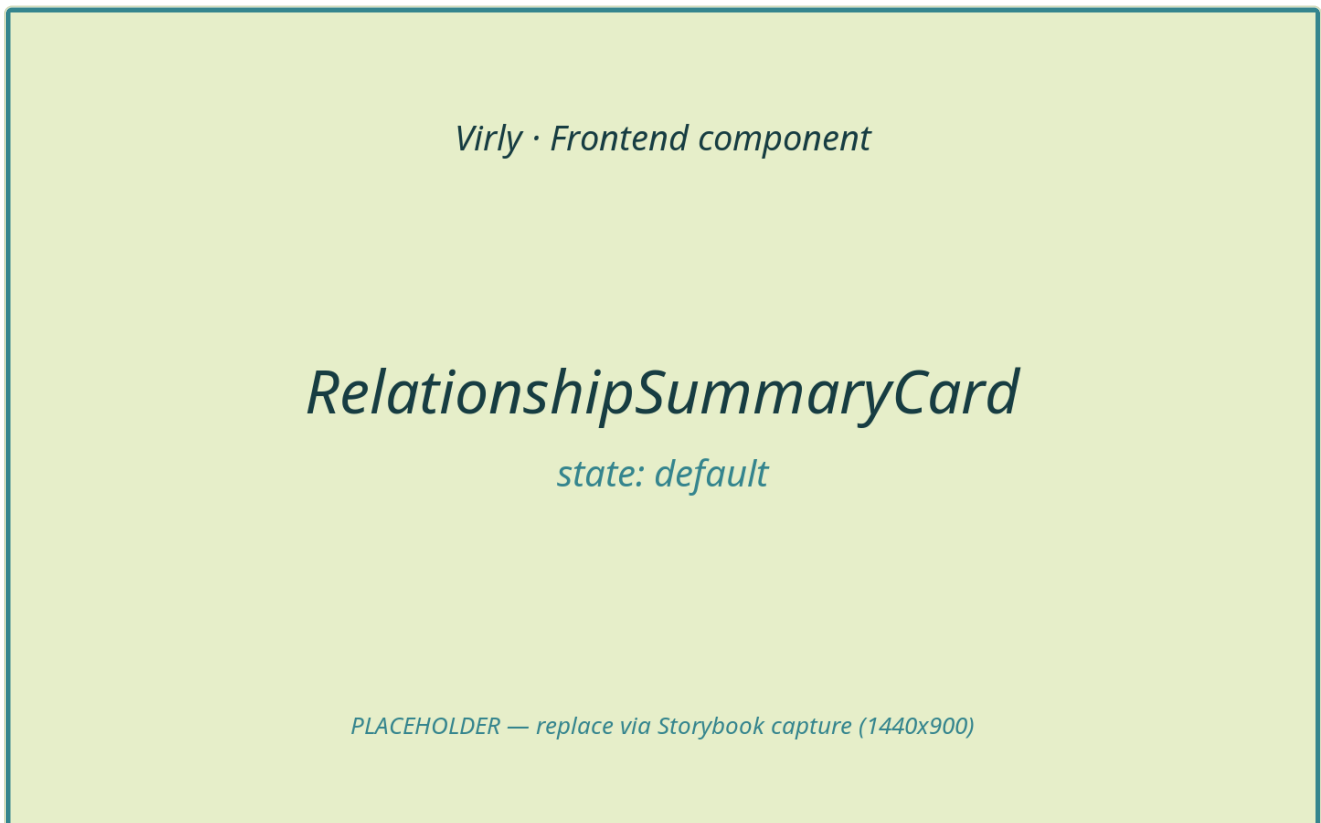
Notes / gotchas

For unverified recipients the copy explicitly tells the user to double-check the email before sending — a deliberate anti-misdirection nudge.

RelationshipSummaryCard

- **Path:** `client/src/features/users/RelationshipSummaryCard.tsx`
- **Category:** feature | **Feature area:** Transfers | **Tier:** Full
- **Summary:** Stats card summarising money sent/received and the net balance between you and the viewed user.

Screenshot(s)



Sent / received / net stats plus transaction count and last interaction.

Architecture constraints

- Backend is authoritative; all totals come from the server relationship summary (`totalSentToUser` , `totalReceivedFromUser` , `netAmount`).
- This card is display-only — it neither initiates nor confirms transfers.
- AI Assistant: N/A.

Purpose & context

Shown on a profile when there is shared history. It presents directional totals (you sent / you received / net) and metadata (transaction count, last interaction date), formatting amounts in the user's display currency.

Anatomy

Section heading (“Between you and {name}”), three stat tiles (sent / received / net with a Scale icon), and a meta list (transactions, last interaction).

Props / API

Prop	Type	Required	Default	Description
relationship	UserRelationshipSummary	Yes	—	Server relationship summary.
viewedName	string	Yes	—	Display name of the viewed user.

State & data

- No local state; uses `useCurrency().formatAmount`.
- Net label is derived from the sign of `relationship.netAmount`.

Interactions & events

None (display-only).

States & variants

- `default` only (rendered solely when `transactionCount > 0`). The “Even” net label appears when net is zero. Loading/empty/error/disabled: N/A.

Dependencies

- Children: `lucide-react` (`ArrowUpRight`, `ArrowDownLeft`, `Scale`).
- Helper: `formatDate`, `useCurrency`.

Accessibility

`aria-label="Activity between you and {name}";` stat icons `aria-hidden`; meta list uses `<dl>/<dt>/<dd>`.

Usage example

```
<RelationshipSummaryCard relationship={relationship} viewedName={user.displayName} />
```

Related / used by

Rendered by `UserProfilePage` alongside `RecentRelationshipTransactions` (documented under Transactions / History).

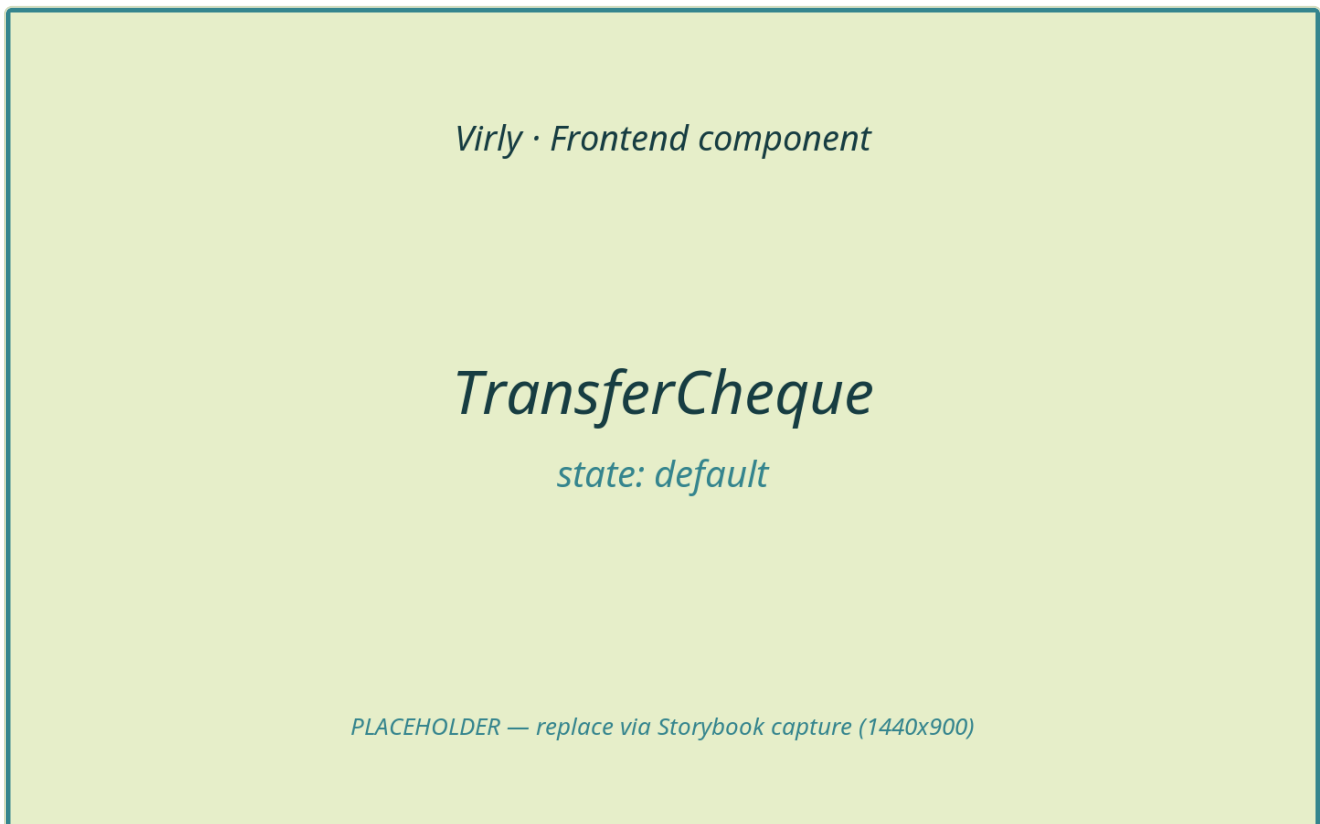
Notes / gotchas

`formatAmount` converts the authoritative ILS figures into the selected display currency; the underlying ledger values stay ILS.

TransferCheque

- **Path:** `client/src/components/TransferCheque.tsx`
- **Category:** form | **Feature area:** Transfers | **Tier:** Full
- **Summary:** The bank-cheque surface that is editable in “form” mode and read-only in “review”/“success” mode (stamped “Cleared” on success).

Screenshot(s)



Form mode: editable payee, amount, currency, and memo.

Virly · Frontend component

TransferCheque

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Form mode with recipient/amount validation errors.

Virly · Frontend component

TransferCheque

state: review

PLACEHOLDER — replace via Storybook capture (1440x900)

Review mode: read-only cheque with a linked payee.

Virly · Frontend component

TransferCheque

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Success mode: "Cleared" stamp animation.

Architecture constraints

- Backend is authoritative; this component holds no balance and moves no money.
- It is the *input surface* for a transfer; the explicit confirmation lives on `TransferPage` ("Sign & send"), which calls `POST /api/transactions`. The cheque itself never calls the API.
- AI Assistant: N/A.

Purpose & context

Virly's signature transfer UI. In `form` mode the payee, amount, currency, and memo are controlled inputs; in `review / success` mode the same values render as static cheque text (the payee becomes a profile link, success stamps "Cleared"). All state is owned by `TransferPage`.

Anatomy

- Watermark + guilloche + (success) "Cleared" stamp.
- Header: brand + cheque number + date.
- Pay line: recipient (input or link) + amount box (currency glyph + figure).
- Amount-in-words line (`amountInWords`).
- Currency `<select>` (form mode only).
- Footer: memo (input/static) + signature line.
- MICR strip.

Props / API

Prop	Type	Required	Default	Description
<code>mode</code>	<code>TransferChequeMode</code> (<code>"form"</code> <code>"review"</code> <code>"success"</code>)	Yes	—	Drives editable vs read-only + the stamp.
<code>chequeNumber</code>	<code>string</code>	Yes	—	Serial shown top-right and in MICR.
<code>issueDate</code>	<code>string</code>	Yes	—	Pre-formatted issue date.
<code>holderEmail</code>	<code>string</code> <code>null</code>	No	—	Source of the signature name.
<code>currency</code>	<code>DisplayCurrency</code>	Yes	—	Glyph, words, and picker value.
<code>payee</code>	<code>string</code>	Yes	—	Finalised recipient (linked in review/success).
<code>recipientEmail</code>	<code>string</code>	Yes	—	Controlled recipient (form mode).
<code>amount</code>	<code>string</code>	Yes	—	Controlled amount string.
<code>reason</code>	<code>string</code>	Yes	—	Controlled / static memo.
<code>errors</code>	<code>TransferChequeErrors</code>	No	<code>{}</code>	<code>recipientEmail</code> / <code>amount</code> / <code>reason</code> messages.
<code>onRecipientEmailChange</code>	(<code>value: string</code>) => <code>void</code>	No	—	Recipient input handler.
<code>onAmountChange</code>	(<code>value: string</code>) => <code>void</code>	No	—	Amount input handler.
<code>onReasonChange</code>	(<code>value: string</code>) => <code>void</code>	No	—	Memo input handler.
<code>onCurrencyChange</code>	(<code>currency: DisplayCurrency</code>) => <code>void</code>	No	—	Currency picker handler.

State & data

- No local state; derives `holderName` , `hasAmount` , words, and a formatted figure. No data fetching.

Interactions & events

- Form mode: input changes call the matching `on*Change` callback.
- Review/success: payee renders as a `Link` to `/users/:payee` .

States & variants

- `default` (form), `error` (form with errors), `review` (read-only), `success` (“Cleared” stamp). Disabled/loading: N/A (the parent disables buttons).

Dependencies

- Libraries: `framer-motion` , `react-router-dom` .
- Helpers: `amountInWords` , currency labels/glyphs. Styling: `cheque-*` (`global.css`).

Accessibility

Inputs are labelled via `aria-label` (e.g. “Amount in Shekels”); error spans render adjacent to fields; `aria-invalid` reflects errors; decorative elements (`watermark` , `guilloche` , `stamp` , `MICR`) are `aria-hidden` . TODO: associate field errors with inputs via `aria-describedby` .

Usage example

```
<TransferCheque
  mode={step}
  chequeNumber={chequeNumber}
  issueDate={issueDate}
  currency={currency}
  payee={payee}
  recipientEmail={recipientEmail}
  amount={amount}
  reason={reason}
  errors={errors}
  onRecipientEmailChange={setRecipientEmail}
  onAmountChange={setAmount}
  onReasonChange={setReason}
  onCurrencyChange={(next) => { setCurrency(next); setQuote(null); }}
/>
```

Related / used by

Rendered by `TransferPage` across all three steps. Pairs with `TransferQuoteSmallPrint` in review mode.

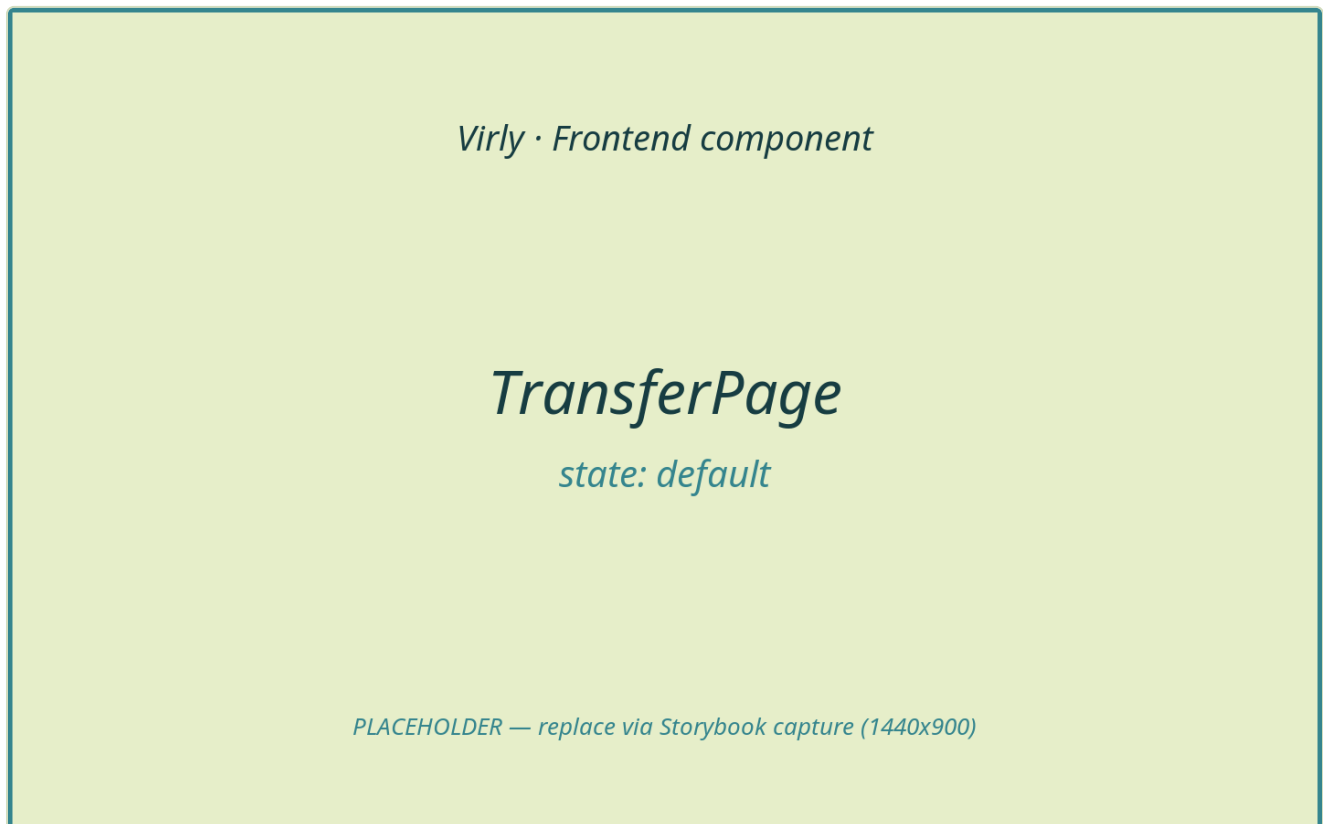
Notes / gotchas

`mode` is the single switch for editable vs static rendering — the same markup serves all three steps, which keeps the cheque visually stable as it “clears”.

TransferPage

- **Path:** `client/src/features/transfer/TransferPage.tsx`
- **Category:** page | **Feature area:** Transfers | **Tier:** Full
- **Summary:** The three-step transfer flow (form → review → success) built around the cheque, with server-quoted FX for non-ILS amounts.

Screenshot(s)



Form step: cheque + recent payees + “Review cheque”.

Virly · Frontend component

TransferPage

state: review

PLACEHOLDER — replace via Storybook capture (1440x900)

Review step: quote small-print + projected balance + “Sign & send”.

Virly · Frontend component

TransferPage

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Success step: cleared cheque + new balance.

Virly · Frontend component

TransferPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Form step with server/validation errors (e.g. insufficient balance).

Architecture constraints

- Backend is authoritative; the page never debits a balance locally — it updates `AuthProvider` only from the server's `response.newBalance`.
- **Explicit confirmation gate:** the transfer executes only on the review step's “Sign & send”, which calls `api.transfer` → `POST /api/transactions`. The form/review steps prepare the request; nothing before “Sign & send” moves money. For non-ILS amounts the review step first fetches an authoritative quote (`POST /api/transactions/quote`); a `409` (rate changed) refreshes the quote and asks the user to confirm the new figure.
- AI Assistant: N/A.

Purpose & context

The primary money-movement flow. It fetches the account summary (balance + recent payees), validates the draft, prepares an FX quote when needed, and on explicit confirmation submits the transfer. The balance side panel projects the post-transfer balance from the authoritative balance and the (quoted or estimated) ILS amount.

Anatomy

- `PageHeader` (“Write a cheque”).

- Cheque panel: `AnimatePresence` over the three steps, each rendering `TransferCheque` plus step-specific chrome (recent-payee chips, quote small-print, projection, success banner, action buttons).
- Balance aside card: current balance + projected after-transfer balance.

Props / API

None.

State & data

- Local state: `recipientEmail`, `amount`, `currency`, `reason`, `errors`, `step` (`"form"` | `"review"` | `"success"`), `isSubmitting`, `summary`, `quote`, `quoteNotice`, `result`.
- Hooks: `useAuth`, `useCurrency`, `useNavigate`, `useEffect`, `useMemo`.
- Data: `api.accountSummary` (mount); `api.transferQuote` (`POST /api/transactions/quote`) on review for non-ILS; `api.transfer` (`POST /api/transactions`) on confirm.

Interactions & events

- `handleReview` → `validate` → (ILS: go to review) / (non-ILS: fetch quote → review).
- `handleSubmit` (Sign & send) → `api.transfer` → `auth.updateBalance` → `success`; `409` → refresh quote + notice; field/ `404` / `400` errors → back to form.
- Recent-payee chip → set recipient.
- Success actions → navigate `/transactions` or reset the form.

States & variants

- `default` / `form`, `loading` ("Preparing quote..." / "Sending..."), `review`, `success`, `error`.
Empty/disabled: buttons disabled while `isSubmitting`.

Dependencies

- Children: `TransferCheque`, `TransferQuoteSmallPrint`, `Button`, `Card`, `ErrorBanner`, `SuccessBanner`, `PageHeader`, `PageStack`, `ResponsiveGrid`.
- Libraries: `framer-motion`, `react-router-dom`.
- Helpers: `validateEmail` / `validateAmount` / `validateReason`, `getQuickContacts`, currency conversion helpers.

Accessibility

Cheque inputs labelled via `TransferCheque`; banners carry `role`; the balance panel exposes the current/projected balance as text. TODO: move focus to the review actions when the step changes.

Usage example

```
<Route path="/transfer" element={<TransferPage />} />
```

Related / used by

Routed inside the protected shell. Entry points that prefill the recipient: `QuickContacts`, `UserProfileHeader` / `RecipientStatusCard` / `EmptyRelationshipState` (via `virly-prefill-recipient`).

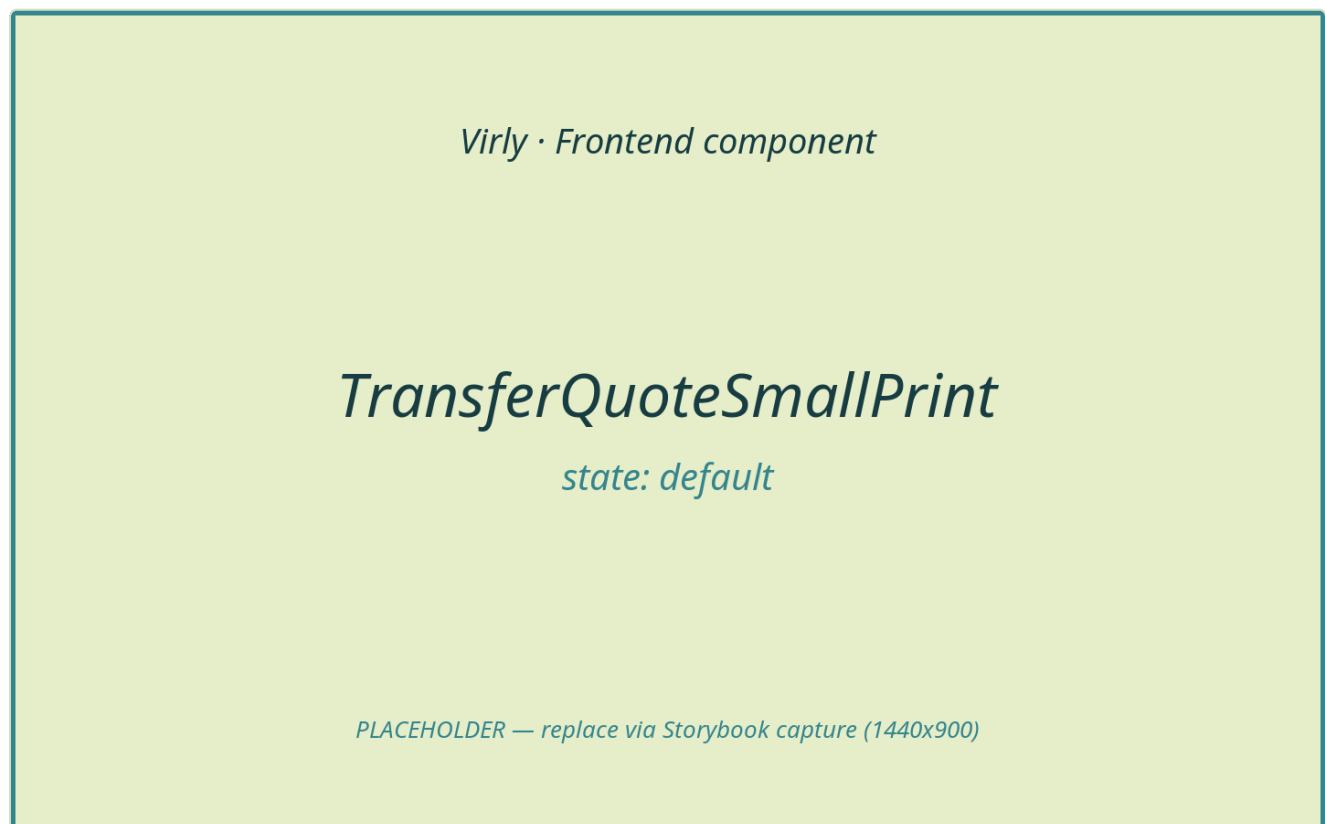
Notes / gotchas

- The self-transfer guard rejects sending to your own email client-side.
- The pre-review ILS estimate is a convenience; the server quote is authoritative and is what is submitted.
- A `409` on submit means the daily FX rate changed between quote and confirm — the user must re-confirm the refreshed amount.

TransferQuoteSmallPrint

- **Path:** `client/src/features/transfer/TransferQuoteSmallPrint.tsx`
- **Category:** feature | **Feature area:** Transfers | **Tier:** Full
- **Summary:** The disclosure line under a non-ILS review showing the actual ILS ledger amount, the rate used, and the rate date.

Screenshot(s)



Actual ILS amount + entered → ILS rate + rate date.

Architecture constraints

- Backend is authoritative; the displayed ILS amount and rate are the server's quote (`TransferQuote`), not a client computation.
- It is informational only; it neither initiates nor confirms the transfer.
- AI Assistant: N/A.

Purpose & context

Renders the small print in the review step so the user sees exactly how much ILS will leave the account and at what rate before confirming. For ILS quotes it renders nothing (no conversion happens).

Anatomy

A single `<p class="transfer-quote-small-print">` with the ILS amount, the `enteredCurrency` → ILS rate, and the rate date.

Props / API

Prop	Type	Required	Default	Description
<code>quote</code>	<code>TransferQuote</code>	Yes	—	Server quote (amount, rate, dates).

State & data

None. Returns `null` when `quote.enteredCurrency === "ILS"`.

Interactions & events

None.

States & variants

- `default` (non-ILS quote shown). For ILS the component renders nothing. Loading/empty/error/success/disabled: N/A.

Dependencies

- Helper: `formatCurrency`.

Accessibility

Plain text paragraph; inherits review-area context.

Usage example

```
{quote ? <TransferQuoteSmallPrint quote={quote} /> : null}
```

Related / used by

Rendered by `TransferPage` in the review step.

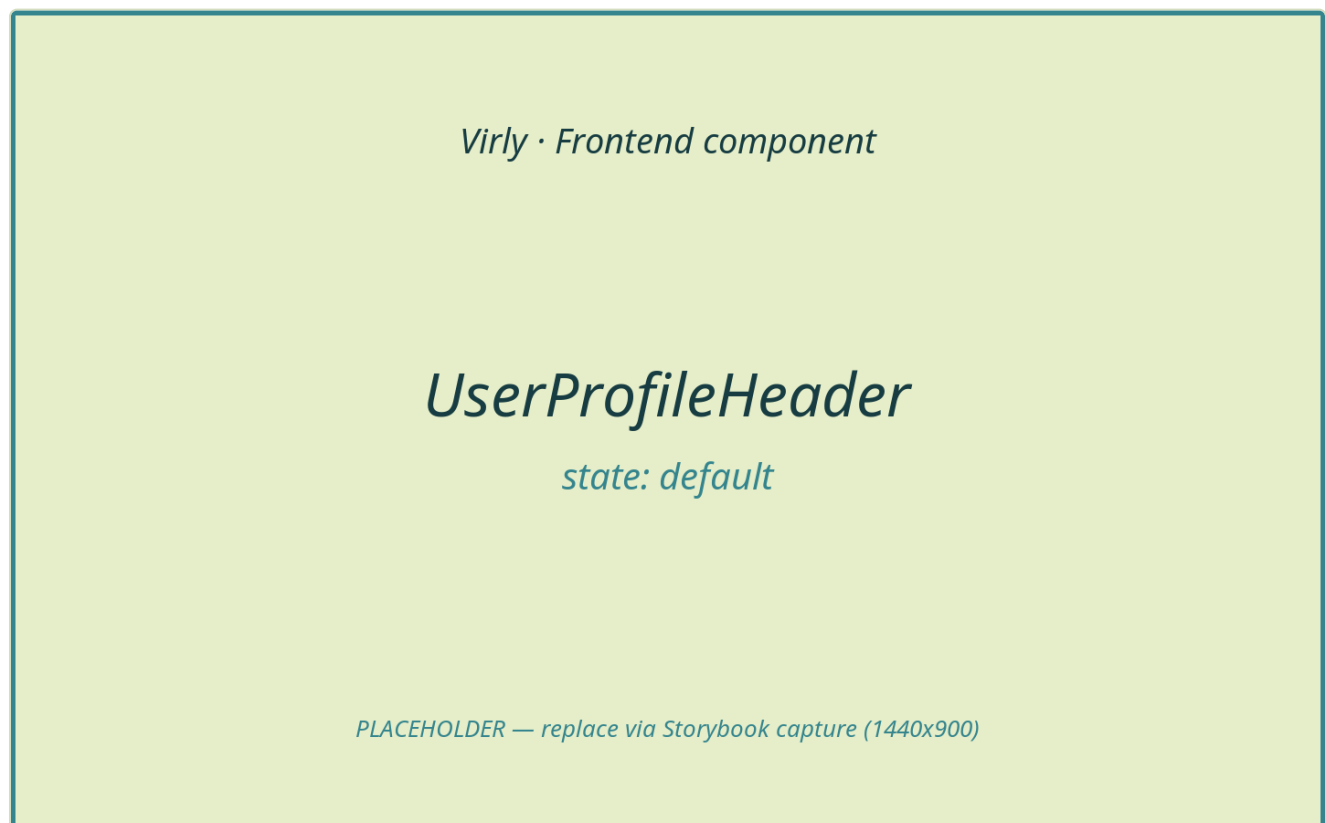
Notes / gotchas

Falls back to `rateFetchedAt` 's date (or "today") when `rateValidForDate` is absent.

UserProfileHeader

- **Path:** `client/src/features/users/UserProfileHeader.tsx`
- **Category:** feature | **Feature area:** Transfers | **Tier:** Full
- **Summary:** Profile masthead: avatar, name + verified badge, email, member since, and a "Transfer" CTA when allowed.

Screenshot(s)



Verified user with a Transfer CTA.

Virly · Frontend component

UserProfileHeader

state: disabled

PLACEHOLDER — replace via Storybook capture (1440x900)

Own profile / not-allowed — no Transfer CTA.

Architecture constraints

- Backend is authoritative; verification + eligibility come from the server.
- The “Transfer” CTA only prefills the recipient and opens the cheque flow; execution still requires confirmation + `POST /api/transactions`.
- AI Assistant: N/A.

Purpose & context

The header of `UserProfilePage`. It identifies the viewed user and offers the primary action — sending money — when `canSendMoney` is true.

Anatomy

Avatar image, identity block (name + optional verified badge, email, member since, “this is your profile” note), optional Transfer `Button`.

Props / API

Prop	Type	Required	Default	Description
<code>user</code>	<code>PublicUserProfile</code>	Yes	—	Public profile (name, email, verified, memberSince).

Prop	Type	Required	Default	Description
<code>isSelf</code>	<code>boolean</code>	Yes	—	Whether this is the viewer's own profile.
<code>canSendMoney</code>	<code>boolean</code>	Yes	—	Whether to show the Transfer CTA.
<code>onSendMoney</code>	<code>() => void</code>	Yes	—	Prefills recipient + navigates to transfer.

State & data

None; formats `memberSince` via `Intl.DateTimeFormat`.

Interactions & events

“Transfer” button → `onSendMoney`.

States & variants

- `default` (CTA present), `disabled` (no CTA — self or not allowed), verified badge variant. Loading/empty/error/success: N/A.

Dependencies

- Children: `Button`, `lucide-react` (`BadgeCheck`, `Send`).
- Helper: `getUserAvatarUrl`.

Accessibility

`aria-label="User profile"`; avatar uses empty `alt` (decorative); badge icon `aria-hidden`.

Usage example

```
<UserProfileHeader
  user={user}
  isSelf={isSelf}
  canSendMoney={relationship.canTransferToUser}
  onSendMoney={handleSendMoney}
/>
```

Related / used by

Rendered by `UserProfilePage`.

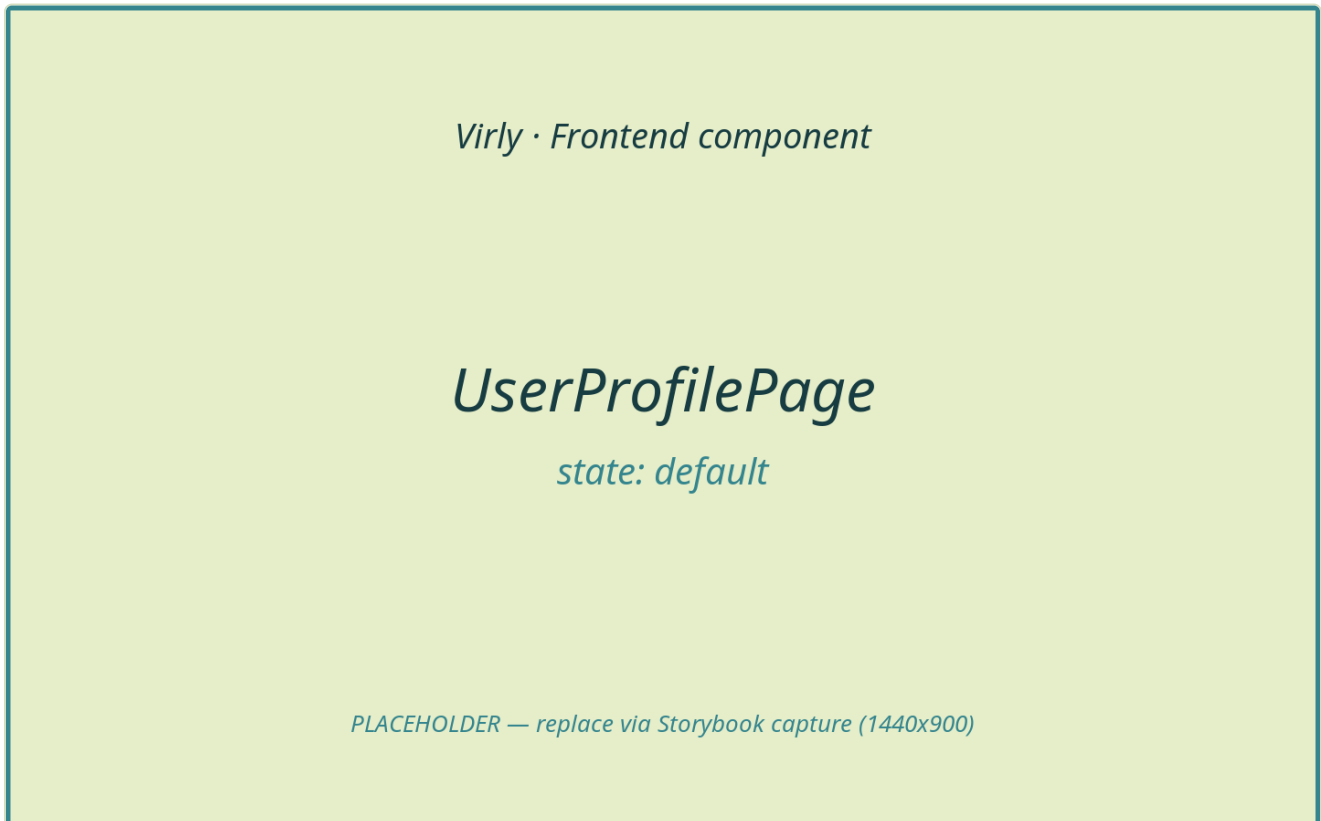
Notes / gotchas

The avatar URL falls back to a generated initial-avatar SVG (`getUserAvatarUrl`) when no avatar service is configured.

UserProfilePage

- **Path:** `client/src/features/users/UserProfilePage.tsx`
- **Category:** page | **Feature area:** Transfers | **Tier:** Full
- **Summary:** Recipient/relationship profile reached from counterparty links; shows verification, shared history, and a transfer CTA.

Screenshot(s)



Other user with shared history + recipient status sidebar.

Virly · Frontend component

UserProfilePage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Skeleton while the profile loads.

Virly · Frontend component

UserProfilePage

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

No shared history (`EmptyRelationshipState`).

Virly · Frontend component

UserProfilePage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Not found (404) / load error states.

Architecture constraints

- Backend is authoritative; the page reads a server profile + relationship and never computes eligibility or balances itself.
- Its only money-related action is the “Transfer” CTA, which prefills the recipient (`virly-prefill-recipient`) and navigates to `TransferPage` . The transfer still requires explicit confirmation + `POST /api/transactions` .
- AI Assistant: N/A.

Purpose & context

Opened from counterparty links across the app (`/users/:userId`). It fetches the public profile + relationship summary, then composes the header, relationship summary, shared transactions, recipient-status sidebar, and the appropriate empty/self branches. The “Transfer” action funnels back into the cheque flow.

Anatomy

- Loading (`Skeleton`), `404` (`EmptyState` , `UserX`), and generic-error (retry) branches.
- `UserProfileHeader` .
- Self branch (account links) **or** history branch (`RelationshipSummaryCard` + `RecentRelationshipTransactions`) **or** `EmptyRelationshipState` .
- Sidebar: `RecipientStatusCard` (non-self).

Props / API

None. (Reads `:userId` from `useParams`.)

State & data

- Local state: `profile`, `error` (`{ status, message }`), `isLoading`, `reloadKey`.
- Hooks: `useParams`, `useNavigate`, `useCallback`, `useEffect`, `useState`.
- Data: `api.userProfile(userId)` → `GET /api/users/:idOrEmail/profile`.

Interactions & events

- Mount / `reloadKey` change → fetch profile.
- “Transfer” (`handleSendMoney`) → store `virly-prefill-recipient` → navigate `/transfer`.
- “Try again” → bump `reloadKey`.

States & variants

- `default` (other user with history), `loading`, `empty` (no history), `error` (`404` vs generic), `self` (own profile). Disabled/success: N/A.

Dependencies

- Children: `UserProfileHeader`, `RelationshipSummaryCard`, `RecentRelationshipTransactions`, `RecipientStatusCard`, `EmptyRelationshipState`, plus `Primitives` (`Card`, `EmptyState`, `ErrorBanner`, `PageHeader`, `PageStack`, `ResponsiveGrid`, `Skeleton`, `Button`).
- Loaded lazily by `App` (`React.lazy` + `Suspense`).

Accessibility

`PageHeader` provides the page heading; the `404` branch uses `EmptyState`; errors use `ErrorBanner`.

Usage example

```
<Route path="/users/:userId" element={<Suspense fallback={<RouteFallback />}><UserProfilePage /></Suspense>} />
```

Related / used by

Linked from `TransactionList`, `TransactionReceipt`, `QuickContacts`, and the review/success cheque payee. Renders the rest of the `features/users/*` cluster.

Notes / gotchas

The `:userId` segment may be an id **or** an email (the API accepts either); counterparty links pass the email.

Transactions / History

This area covers viewing past activity: the filterable transactions page, the reusable transaction list with pagination, the receipt dialog, and the profile-scoped “shared activity” container. All amounts are formatted in the user’s display currency but the underlying ledger figures are the server’s authoritative ILS values; nothing here mutates state. Screenshots are placeholders pending Storybook capture.

Components in this area

- [RecentRelationshipTransactions](#)
- [TransactionDetailsDialog](#)
- [TransactionList](#)
- [TransactionReceipt](#)
- [TransactionsPage](#)

RecentRelationshipTransactions

- **Path:** `client/src/features/users/RecentRelationshipTransactions.tsx`
- **Category:** hook-bound container | **Feature area:** Transactions / History | **Tier:** Full
- **Summary:** “Shared activity” list on a user profile — paginates the transactions between you and the viewed user and opens the receipt dialog.

Screenshot(s)

Virly · Frontend component

RecentRelationshipTransactions

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Initial shared transactions with a “View all” affordance.

Virly · Frontend component

RecentRelationshipTransactions

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Skeleton while a page loads.

RecentRelationshipTransactions

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Inline error when shared transactions fail to load.

Purpose & context

Rendered by `UserProfilePage` (Transfers area) when there is shared history. It seeds from the profile response's `recentTransactions`, then lazily paginates via the `relationship-transactions` endpoint when the user pages or expands. Each row is reshaped into the viewer-relative `Transaction` shape the shared receipt dialog expects.

Anatomy

- Section heading (“Shared activity”) + inline error.
- `TransactionRows` (direction icon, “Sent to / Received from {name}”, amount, status).
- Pagination nav, or a “View all {n} transactions” expander.
- `TransactionDetailsDialog` for the selected row.

Props / API

Prop	Type	Required	Default	Description
<code>idOrEmail</code>	<code>string</code>	Yes	—	The viewed user's id or email.
<code>initialTransactions</code>	<code>RelationshipTransaction[]</code>	Yes	—	Seed rows from the profile response.
	<code>number</code>	Yes	—	

Prop	Type	Required	Default	Description
<code>totalCount</code>				Total shared transactions (drives "View all").
<code>viewedName</code>	<code>string</code>	Yes	—	Display name for row copy.
<code>viewedEmail</code>	<code>string</code>	No	—	Counterparty email for the receipt.

State & data

- Local state: `page` (`number | null`), `transactions`, `pagination`, `isLoading`, `error`, `selectedTransaction`.
- Hooks: `useState`, `useEffect`, `useCurrency`.
- Data: `api.userRelationshipTransactions(idOrEmail, page, 10)` → `GET /api/users/:idOrEmail/transactions?page&limit`.

Interactions & events

- "View all {n}" → `setPage(1)` (triggers the first server fetch).
- Pagination → `setPage`.
- Row click/Enter/Space → `setSelectedTransaction` (mapped via `toDialogTransaction`).

States & variants

- `default` (seed rows), `loading` (`Skeleton`), `error` (inline), paginated/expanded variants. Empty: when there is no history the parent renders `EmptyRelationshipState` instead. Success/disabled: N/A.

Dependencies

- Children: `TransactionDetailsDialog`, `Button`, `Skeleton`, `lucide-react`.
- Helper: `formatRelativeDate`, `useCurrency`.

Accessibility

`aria-label="Shared transactions with {name}";` rows are `role="button"`, `tabIndex={0}`, keyboard-activatable; pagination nav is labelled.

Usage example

```
<RecentRelationshipTransactions
  idOrEmail={userId}
  initialTransactions={recentTransactions}
  totalCount={relationship.transactionCount}
  viewedName={user.displayName}
  viewedEmail={user.email}
/>
```

Related / used by

Rendered by `UserProfilePage` (Transfers area). Reuses `TransactionDetailsDialog` / `TransactionReceipt`.

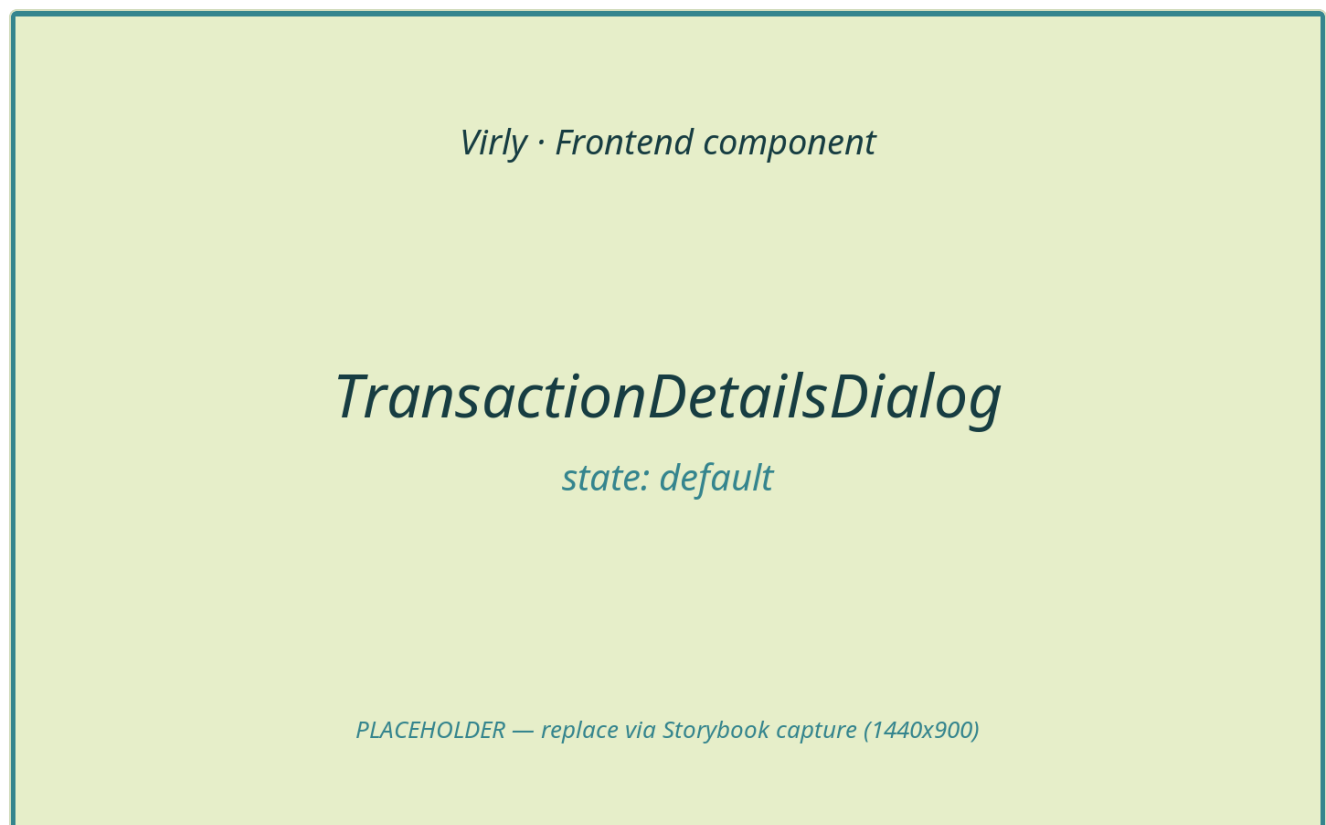
Notes / gotchas

Relationship transactions carry a positive `amount` + `direction`; the container signs the amount (`received` → positive, `sent` → negative) before handing it to the shared receipt.

TransactionDetailsDialog

- **Path:** `client/src/components/TransactionDetailsDialog.tsx`
- **Category:** modal/overlay | **Feature area:** Transactions / History | **Tier:** Full
- **Summary:** Accessible modal overlay that hosts the `TransactionReceipt` for a selected transaction, with focus trapping and scroll lock.

Screenshot(s)



Open dialog with the receipt centred over a dimmed page.

Purpose & context

The shared “view this transaction” overlay used by the dashboard, transactions page, and profile shared-activity list. It opens whenever a non-null `transaction` is passed, traps focus, locks body

scroll, closes on Escape / backdrop click, and restores focus to the previously-focused element on close.

Anatomy

- Backdrop (`role="presentation"` , closes on click).
- Dialog (`role="dialog"` , `aria-modal` , `aria-label="Transaction details"`) containing `TransactionReceipt` .

Props / API

Prop	Type	Required	Default	Description
<code>transaction</code>	<code>Transaction null</code>	Yes	—	The transaction to show; <code>null</code> closes the dialog.
<code>onClose</code>	<code>() => void</code>	Yes	—	Called on Escape, backdrop click, or receipt actions.

State & data

- Refs only (`dialogRef` , `previouslyFocusedRef`). No data fetching.

Interactions & events

- Open → focus first focusable; lock `body` scroll.
- Escape → `onClose` ; Tab/Shift+Tab → wrap focus within the dialog.
- Backdrop click → `onClose` ; inner click stops propagation.
- Close → restore previous focus + scroll.

States & variants

- `default` (open). Closed renders nothing (`AnimatePresence` exit). Loading/empty/error/success/disabled: N/A.

Dependencies

- Children: `TransactionReceipt` .
- Libraries: `framer-motion` .

Accessibility

Full modal semantics: `role="dialog"` , `aria-modal="true"` , focus trap, Escape to close, focus restoration, body scroll lock. A strong reference example for the rest of the app.

Usage example

```
<TransactionDetailsDialog
  transaction={selectedTransaction}
  onClose={() => setSelectedTransaction(null)}
/>
```

Related / used by

Used by `DashboardPage`, `TransactionsPage`, `AccountStatement`, and `RecentRelationshipTransactions`. Wraps `TransactionReceipt`.

Notes / gotchas

The focus trap re-focuses the dialog itself if no focusable children exist, preventing focus escaping to the page behind.

TransactionList

- **Path:** `client/src/components/TransactionList.tsx`
- **Category:** feature | **Feature area:** Transactions / History | **Tier:** Full
- **Summary:** The reusable transaction list: directional rows, counterparty links, amounts, optional pagination, and an empty state.

Screenshot(s)

Virly · Frontend component

TransactionList

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Rows with credit/debit marks, counterparty links, and pagination.

Virly · Frontend component

TransactionList

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

Empty state with a "Transfer" CTA.

Purpose & context

Renders a list of `Transaction`s with credit/debit styling. Each row links the counterparty to their profile and (when selectable) opens the receipt dialog. It supports compact mode and optional pagination controls.

Anatomy

- Empty branch: `EmptyState` + transfer CTA.
- Per row: direction mark, counterparty link + reason/relative date, signed amount + "Completed".
- Optional pagination nav (Previous / page indicator / Next).

Props / API

Prop	Type	Required	Default	Description
<code>transactions</code>	<code>Transaction[]</code>	Yes	—	Rows to render.
<code>pagination</code>	<code>Pagination</code>	No	—	Enables the pager (with <code>page</code> / <code>onPageChange</code>).
<code>page</code>	<code>number</code>	No	—	Current page (for the pager).
		No	—	Page change handler.

Prop	Type	Required	Default	Description
onPageChange	(page: number) => void			
compact	boolean	No	false	Compact spacing variant.
onTransactionSelect	(transaction: Transaction) => void	No	—	Makes rows selectable (opens the dialog).

State & data

- No local state; uses `useCurrency().formatAmount`.

Interactions & events

- Row click/Enter/Space (when `onTransactionSelect`) → select; clicks on the inner counterparty link are excluded from selection.
- Pager → `onPageChange`.

States & variants

- `default`, `empty`, `compact`, paginated. Loading is owned by the parent. Error/success/disabled: N/A (pager buttons disable at bounds).

Dependencies

- Children: `Button`, `EmptyState`, `Link`, `lucide-react`.
- Helper: `formatRelativeDate`, `useCurrency`.

Accessibility

Selectable rows are `role="button"`, `tabIndex={0}`, keyboard-activatable; counterparty links carry `aria-label`; pager nav is labelled. Direction marks are `aria-hidden`.

Usage example

```
<TransactionList
  transactions={response?.transactions ?? []}
  pagination={response?.pagination}
  page={page}
  onPageChange={setPage}
  onTransactionSelect={setSelectedTransaction}
/>
```

Related / used by

Rendered by `TransactionsPage`. (The dashboard uses `AccountStatement` instead.)

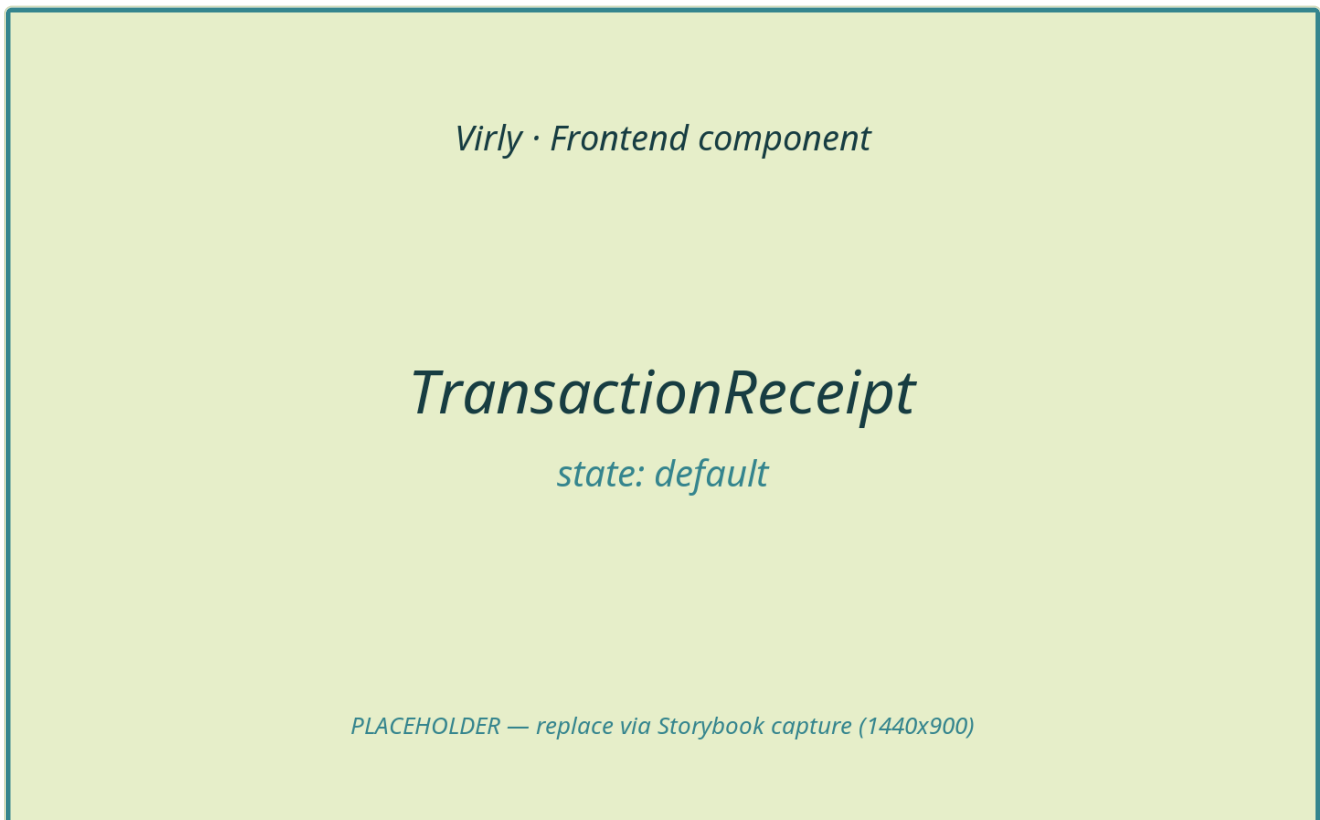
Notes / gotchas

A positive `amount` is treated as a credit (received); negative as a debit (sent). The “Completed” label is static — the list only renders settled ledger entries.

TransactionReceipt

- **Path:** `client/src/components/TransactionReceipt.tsx`
- **Category:** feature | **Feature area:** Transactions / History | **Tier:** Full
- **Summary:** A printed-receipt rendering of a single transaction (amount, parties, memo, auth code, barcode) with an animated “Paid/Received” stamp.

Screenshot(s)



Receipt for a debit (“Paid”) with rows + barcode.

TransactionReceipt

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Credit variant ("Received") stamp.

Purpose & context

The content of `TransactionDetailsDialog`. It presents a transaction as a paper receipt: hero amount, "money sent/received" direction, a rows section (from/to, memo, date, optional FX "entered as", auth code), a totals block (no fees), a deterministic barcode derived from the transaction id, and actions (Done / View profile).

Anatomy

- Close button + animated paper (`paper / line / stamp` variants).
- Stamp ("Paid"/"Received"), merchant header, hero amount, rows, totals, barcode, footer, actions.

Props / API

Prop	Type	Required	Default	Description
<code>transaction</code>	<code>Transaction</code>	Yes	—	The transaction to render.
<code>onClose</code>	<code>() => void</code>	Yes	—	Closes the dialog (Done / View profile).

State & data

- `useMemo` for the deterministic barcode widths; `useCurrency().formatAmount`. No data fetching.

Interactions & events

- “Done” → `onClose`.
- “View profile” → `/users/:counterparty` (also calls `onClose`).

States & variants

- `default` (debit “Paid”), `success` (credit “Received”), FX variant (shows “Entered as”).
Loading/empty/error/disabled: N/A.

Dependencies

- Libraries: `framer-motion`, `react-router-dom`, `lucide-react`.
- Helper: `formatDate`, `useCurrency`.

Accessibility

Close button has `aria-label`; decorative paper/stamp/barcode are `aria-hidden`. The receipt is mounted inside the dialog’s focus trap. TODO: ensure the amount and parties are announced (currently visual text).

Usage example

```
<TransactionReceipt transaction={transaction} onClose={onClose} />
```

Related / used by

Rendered by `TransactionDetailsDialog`.

Notes / gotchas

The barcode is derived from the transaction id, so every receipt prints a unique but reproducible pattern. Fees are always shown as zero (“No fees, ever”).

TransactionsPage

- **Path:** `client/src/features/transactions/TransactionsPage.tsx`
- **Category:** page | **Feature area:** Transactions / History | **Tier:** Full
- **Summary:** The full transaction history page with a counterparty filter and server pagination.

Screenshot(s)

Virly · Frontend component

TransactionsPage

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Filter bar + paginated transaction list.

Virly · Frontend component

TransactionsPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Skeleton while a page loads.

Virly · Frontend component

TransactionsPage

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

No results for the current filter.

Virly · Frontend component

TransactionsPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Load error (`ErrorBanner`).

Purpose & context

The dedicated history view. It fetches a page of transactions (optionally filtered by counterparty email), renders them via `TransactionList`, and opens the receipt dialog on row selection. Filtering and paging re-fetch from the server.

Anatomy

- `PageHeader` ("Transactions").
- Filter card: counterparty `Field` + Filter / Reset buttons.
- Error banner.
- List card: `Skeleton` while loading, else `TransactionList`.
- `TransactionDetailsDialog`.

Props / API

None.

State & data

- Local state: `response`, `page`, `counterparty`, `activeCounterparty`, `error`, `filterError`, `isLoading`, `selectedTransaction`.
- Hooks: `useState`, `useEffect`.
- Data: `api.transactions({ page, limit, counterparty })` → `GET /api/transactions?page&limit&counterparty`.

Interactions & events

- Filter submit → `validateEmail` (if filled) → set active filter + reset to page 1.
- Reset → clear filter + page.
- Row select → open dialog.
- Pager (in `TransactionList`) → `setPage`.

States & variants

- `default`, `loading` (`Skeleton`), `empty` (list empty state), `error`. Filter validation error shows under the field. Success/disabled: N/A.

Dependencies

- Children: `TransactionList`, `TransactionDetailsDialog`, plus `Primitives` (`Button`, `Card`, `ErrorBanner`, `Field`, `PageHeader`, `PageStack`, `Skeleton`).
- Helper: `validateEmail`.

Accessibility

Filter is a labelled `Field`; the list and pager provide their own semantics; errors use `ErrorBanner`.

Usage example

```
<Route path="/transactions" element={<TransactionsPage />} />
```

Related / used by

Routed inside the protected shell. Reached from nav, the dashboard “View all”, and post-transfer success.

Notes / gotchas

The filter normalises the counterparty email server-side; an invalid email is caught client-side before the request is issued.

AI Assistant

Virly's assistant is a LangGraph/OpenAI layer behind the backend. On the frontend it is a **read-only / preparation surface**: a floating chat widget that streams the assistant's structured response and renders it as typed cards. The assistant cannot pick tools, move money, or approve a transfer on its own — those decisions live on the server, and any money movement requires an explicit user confirmation that the client posts to a dedicated confirmation endpoint.

Architecture, in one line: the chat UI sends the user's message, displays whatever structured blocks the backend returns, and — for a prepared transfer — shows a confirmation card whose **Confirm** button is the only path that executes the transfer (`POST /api/ai/confirmations/:id` with `action: "confirm"` and the confirmation `version`). The assistant proposes; the user disposes; the backend executes. Every component here carries the **Architecture constraints** callout.

Screenshots are placeholders pending Storybook capture; filenames follow the final convention.

Components in this area

- [AssistantBlocks](#)
- [ChatWidgetDemo](#)
- [FloatingChatWidget](#)

AssistantBlocks

- **Path:** `client/src/components/assistant/AssistantBlocks.tsx`
- **Category:** feature | **Feature area:** AI Assistant | **Tier:** Full
- **Summary:** The renderer that turns the backend's typed `responseBlocks` into cards (account summary, transaction lists, transfer quote/limits/status, pending transfers, transfer confirmation, notices, video CTA).

Screenshot(s)

Virly · Frontend component

AssistantBlocks

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

A mix of rendered block cards (account summary + transaction list).

Virly · Frontend component

AssistantBlocks

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Transfer confirmation block with Confirm / Deny actions.

Virly · Frontend component

AssistantBlocks

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

Empty-state / notice block.

Architecture constraints

- Backend is authoritative; this component does not mutate balances client-side — it only renders the server's structured output.
- For transfers: the `transfer_confirmation` block exposes **Confirm/Deny** that call the parent's `onConfirmTransfer` / `onDenyTransfer` callbacks; the actual execution (`POST /api/ai/confirmations/:id`) is performed by `FloatingChatWidget` . Buttons are disabled once the confirmation is no longer `pending` or has expired.
- AI Assistant: this is a read-only render of prepared actions; the assistant cannot select tools or approve a transfer — the user must press Confirm.

Purpose & context

A pure presentational dispatcher: given `blocks: AssistantResponseBlock[]` , it maps each block `type` to a typed card. It is the structured (v1) rendering path for assistant responses; free text is rendered separately by `AssistantMarkdown` (exported from the same file). It is bidi-aware (Hebrew/RTL detection) and formats money via the assistant money helpers.

Anatomy (block renderers)

- `text` , `account_summary` (`AccountSummaryCard`), `transaction_list` (`TransactionListCard` + `TransactionRow`), `transaction_detail` , `transaction_stats` , `pending_transfers` (`PendingTransferCard`), `transfer_quote` , `transfer_confirmation`

(`TransferConfirmationCard`), `transfer_status`, `transfer_limits`, `video_session_cta` (`VideoSessionCtaCard`), `empty_state`, `notice`.

- Shared primitives: `AssistantCard`, `KeyValueGrid`, `MoneyValue`, `DateTimeValue`, `CounterpartyValue`, `StatusBadge`.
- Helpers: `hasTransferConfirmationBlock`, `AssistantMarkdown` (also exported).

Props / API

Prop	Type	Required	Default	Description
<code>blocks</code>	<code>AssistantResponseBlock[]</code>	Yes	—	The typed blocks to render (returns <code>null</code> if empty).
<code>locale</code>	<code>string</code>	No	—	Preferred locale for money/ date formatting.
<code>confirmationStatus</code>	<code>TransferConfirmationCardStatus</code>	No	—	Drives the confirmation card's button/label state.
<code>onConfirmTransfer</code>	<code>(confirmation: AiTransferConfirmation) => void</code>	No	—	Fired by the Confirm button.
<code>onDenyTransfer</code>	<code>(confirmation: AiTransferConfirmation) => void</code>	No	—	Fired by the Deny button.

State & data

- No local state; no data fetching. Money formatting via `formatMoney` / `formatMoneyILS` ; bidi via a Hebrew-range regex.

Interactions & events

- `transfer_confirmation` Confirm → `onConfirmTransfer(confirmation)`.
- `transfer_confirmation` Deny → `onDenyTransfer(confirmation)`.
- `video_session_cta` → `Link` to `block.appPath`.
- Buttons are disabled when `confirmationStatus !== "pending"` or the confirmation has expired.

States & variants

- `default` (informational blocks), `success` (confirmation card actionable), `empty` (`empty_state` / `notice`), expired/superseded confirmation (disabled actions). Loading is owned by the widget. Error: surfaced as `notice` blocks.

Dependencies

- Libraries: `lucide-react`, `react-router-dom`.
- Helpers: `formatDate`, `formatMoneyILS`, `cn`. Styling: Tailwind utility classes.

Accessibility

Cards use `dir="auto"` / explicit `dir` for bidi; money/dates are wrapped in `<bdi dir="ltr">` to keep numerals LTR inside RTL text; status badges are text. Confirm/Deny are real `<button>`s with disabled states. TODO: add `aria-label`s to the Confirm/Deny buttons (currently rely on visible text + icon).

Usage example

```
<AssistantBlocks
  blocks={chatMessage.responseBlocks}
  confirmationStatus={chatMessage.confirmationStatus}
  onConfirmTransfer={(c) => handleConfirmationAction(chatMessage.id, c, "confirm")}
  onDenyTransfer={(c) => handleConfirmationAction(chatMessage.id, c, "deny")}
/>
```

Related / used by

Rendered by `FloatingChatWidget`. `AssistantMarkdown` (same file) renders the assistant's free-text message.

Notes / gotchas

`hasTransferConfirmationBlock` lets the widget avoid rendering its *own* legacy pending-transfer card when a `transfer_confirmation` block is already present — the two confirmation UIs are mutually exclusive per message.

ChatWidgetDemo

- **Path:** `client/src/components/ui/floating-chat-widget-demo.tsx`
- **Category:** page (demo harness) | **Feature area:** AI Assistant | **Tier:** Full
- **Summary:** A minimal demo/Storybook harness that centres a single `FloatingChatWidget` on a blank page.

Screenshot(s)

Virly · Frontend component

ChatWidgetDemo

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

The widget mounted alone on a blank background.

Architecture constraints

- Backend is authoritative; this harness adds no logic — all behaviour and the confirmation gate live in `FloatingChatWidget`.
- For transfers / AI Assistant: identical constraints to `FloatingChatWidget`; the demo simply mounts it.

Purpose & context

A throwaway wrapper (`export default function Demo`) used to preview the chat widget in isolation (e.g. a future Storybook story). It is **not** part of the production tree — the production widget is mounted by `AppShell`.

Anatomy

A centred flex container rendering one `FloatingChatWidget`.

Props / API

None.

State & data

N/A.

Interactions & events

N/A (delegated to `FloatingChatWidget`).

States & variants

- `default` only. Loading/empty/error/success/disabled: N/A.

Dependencies

- Children: `FloatingChatWidget`.

Accessibility

N/A beyond the widget's own semantics.

Usage example

```
import Demo from "@components/ui/floating-chat-widget-demo";  
// renders a single, centred FloatingChatWidget
```

Related / used by

Standalone harness for `FloatingChatWidget`. Not routed by `App`.

Notes / gotchas

Because it relies on `useAuth`, this harness still needs an `AuthProvider` ancestor to render the widget without throwing.

FloatingChatWidget

- **Path:** `client/src/components/ui/floating-chat-widget-shadcnui.tsx`
- **Category:** hook-bound container | **Feature area:** AI Assistant | **Tier:** Full
- **Summary:** The floating assistant: a launcher button + chat window with agent selection, streamed responses, clarification options, and the transfer confirmation gate.

Screenshot(s)

Virly · Frontend component

FloatingChatWidget

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Open chat with the agent greeting.

Virly · Frontend component

FloatingChatWidget

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Streaming — typing dots + phase label (e.g. "Checking account facts").

Virly · Frontend component

FloatingChatWidget

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Pending-transfer confirmation card with Confirm / Deny.

Virly · Frontend component

FloatingChatWidget

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Assistant unreachable / confirmation failed message.

Architecture constraints

- Backend is authoritative; the widget updates the balance only from the server's response `newBalance` after a confirmed transfer — never locally.
- **Confirmation gate:** a transfer prepared by the assistant executes only when the user presses **Confirm**, which calls `api.aiConfirmation(id, "confirm", version) → POST /api/ai/confirmations/:id` (with an `Idempotency-Key`). **Deny** posts `action: "deny"`. The assistant never auto-confirms.
- AI Assistant: the widget surfaces read-only output and prepared actions; the assistant cannot select tools or approve transfers. Selecting an “agent” only changes the persona, not the permissions.

Purpose & context

The production assistant surface, mounted globally by `AppShell`. It manages the conversation, picks streaming vs non-streaming based on browser support, renders each message (user bubbles, assistant markdown + `AssistantBlocks`, clarification options, and a legacy pending-transfer card), and owns the confirmation gate that executes or denies an assistant-prepared transfer.

Anatomy

- Launcher `motion.button` (open/close).
- Chat window: header with an agent `Select` (Oshri / Chaya / Yehuda / Yohai), message log (`role="log"`, `aria-live="polite"`), streaming indicator with a phase label, and a composer (auto-resizing `textarea` + send button).
- Per assistant message: `AssistantMarkdown`, `AssistantBlocks`, `ClarificationOptions`, and a fallback `PendingTransferCard` (only when there is a confirmation but no `transfer_confirmation` block).

Props / API

None. (Self-contained; reads the user from `useAuth`.)

State & data

- Local state: `isOpen`, `selectedAgent` (`AssistantId`), `message`, `conversationId`, `chatMessages` (`ChatMessage[]`), `isSending`, `streamPhase`.
- Hooks: `useAuth`, `useId`, `useRef`, `useEffect`, `useCallback`, `useState`.
- Data: `api.aiChatStream` (`POST /api/ai/chat/stream`, SSE) or `api.aiChat` (`POST /api/ai/chat`); `api.aiConfirmation` (`POST /api/ai/confirmations/:id`).

Interactions & events

- Submit / Enter (no Shift) → `sendChatMessage` → stream/POST → append response; a `supersededConfirmationId` marks the older card `superseded`.
- Clarification option → `sendChatMessage(value)`.

- Confirm/Deny (from `AssistantBlocks` or the fallback card) → `handleConfirmationAction` → `api.aiConfirmation` → update card status; on `confirmed`, `auth.updateBalance(newBalance)`.

States & variants

- `default` (open), `loading` (`isSending` — dots + phase label), `success` (confirmed transfer), `error` (unreachable / failed confirmation), `closed` (launcher only), `disabled` (send disabled while empty/sending). Empty: greeting only.

Dependencies

- Children: `AssistantBlocks`, `AssistantMarkdown`, `Avatar`, `Button`, `Select`, `ClarificationOptions`, `PendingTransferCard`.
- Libraries: `framer-motion`, `lucide-react`.
- Helpers: `api`, `ApiError`, `supportsAiChatStreaming`, `formatCurrency`, `getDisplayName` / `getInitial` / `getUserAvatarUrl`, `cn`.

Accessibility

Launcher has `aria-label` / `aria-expanded`; the message log is `role="log"`, `aria-live="polite"`; the agent select has an `aria-label`; close has an `aria-label`. Agent personas/greetings are Hebrew (`dir="auto"`). TODO: trap focus within the open window and return focus to the launcher on close.

Usage example

```
// AppShell.tsx
<FloatingChatWidget />
```

Related / used by

Mounted by `AppShell` (present on every protected page). Renders `AssistantBlocks`. Previewed in isolation by `ChatWidgetDemo`.

Notes / gotchas

- The four agents (Oshri, Chaya, Yehuda, Yohai) are personas only; switching does not change capabilities or permissions.
- `aiConfirmation` always sends a fresh `Idempotency-Key`, so a double-click cannot double-execute a transfer.
- Streaming gracefully degrades to a single POST when `ReadableStream` / `TextDecoder` are unavailable.

Layout / Navigation

This area is the app frame: the router and route table, the authenticated shell (sidebar + topbar + animated outlet + mobile nav), the route guards that gate protected vs guest routes, and the 404 surface. None of these move money; they compose the surfaces that do. Screenshots are placeholders pending Storybook capture.

The app entry point `client/src/main.tsx` (mounts `BrowserRouter` → `AuthProvider` → `App`) is a non-visual bootstrap file; see Appendix B in the index.

Components in this area

- [App](#)
- [AppShell](#)
- [NotFoundPage](#)
- [NotFoundSlip](#)
- [RouteGuards](#)
- [ShellTopbar](#)
- [UserProfileSidebar](#)

App

- **Path:** `client/src/app/App.tsx`
- **Category:** layout (router) | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** The route table: public auth routes, the protected shell branch (currency provider + `AppShell`), lazy routes, and the catch-all 404.

Screenshot(s)

Virly · Frontend component

App

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

The app frame: shader background + routed content layer.

Purpose & context

Defines all routes and the global frame. It renders the `ShaderBackground` and a content layer, configures `MotionConfig` `reducedMotion="user"`, and splits routes into guest routes (`/login`, `/register`), open routes (`/verify`, `/resend-verification`), and a protected branch wrapping `CurrencyProvider` + `AppShell` (dashboard, transfer, transactions, users, video, agent queue, settings). `/` redirects to `/dashboard`; `*` renders the 404.

Anatomy

- `MotionConfig` + `ShaderBackground` + `.app-content-layer`.
- Routes : `GuestRoute` -wrapped auth pages; open verify pages; a `ProtectedRoute` → `CurrencyProvider` → `AppShell` layout route with child routes; lazy `UserProfilePage` / `VideoSessionPage` / `AgentVideoSessionsPage` under `Suspense`; redirect + catch-all.

Props / API

None.

State & data

- `useLocation` (passed to `Routes`). No data fetching.

Interactions & events

Routing only. Lazy routes use `Suspense` with a `RouteFallback` (`Skeleton`).

States & variants

- `default` (renders the matched route). Loading: `lazy-route` `Suspense` fallback. Empty/error/success/disabled: N/A.

Dependencies

- Children: `AppShell`, `RouteGuards`, `CurrencyProvider`, all page components, `ShaderBackground`, `Skeleton`.
- Libraries: `react-router-dom`, `framer-motion`.

Accessibility

`MotionConfig` `reducedMotion="user"` makes the whole app honour the user's reduced-motion preference.

Usage example

```
// main.tsx
<BrowserRouter><AuthProvider><App /></AuthProvider></BrowserRouter>
```

Related / used by

Mounted by `main.tsx`. Hosts every page.

Notes / gotchas

`CurrencyProvider` wraps only the protected branch, so currency conversion is available inside the shell but not on the auth pages.

AppShell

- **Path:** `client/src/components/AppShell.tsx`
- **Category:** layout | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** The authenticated chrome: sidebar, topbar, animated route outlet, mobile nav, and the globally-mounted chat widget.

Screenshot(s)

Virly · Frontend component

AppShell

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Sidebar + topbar + content, with the chat launcher.

Virly · Frontend component

AppShell

state: disabled

PLACEHOLDER — replace via Storybook capture (1440x900)

Collapsed-sidebar variant.

Purpose & context

The layout route element for every protected page. It builds the nav items (with an extra “Queue” item for agent roles), renders the `UserProfileSidebar`, `ShellTopbar`, the animated `AnimatedOutlet`, the mobile nav, and the always-on `FloatingChatWidget`. It persists the sidebar collapsed state and plays an entrance animation when arriving from the auth flow.

Anatomy

- `motion.aside` → `UserProfileSidebar` (user, nav items, logout, collapse).
- `shell-main` → `ShellTopbar` + `main.page-frame` → `AnimatePresence` over `AnimatedOutlet` (frozen outlet for exit animation).
- `nav.mobile-nav` (mobile tab bar).
- `FloatingChatWidget`.

Props / API

None.

State & data

- Local state: `isSidebarCollapsed` (persisted to `localStorage` via `virly-sidebar-collapsed`).
- Hooks: `useAuth`, `useLocation`, `useNavigate`, `useNavigationType`, `useOutlet`, `useState`, `useEffect`.
- Data: none directly (reads `auth.user`).

Interactions & events

- Nav links → route changes; forward navigation scrolls to top (POP keeps browser scroll restoration).
- Collapse toggle → persist preference.
- Logout → `auth.logout()` → `navigate /login`.

States & variants

- `default` (expanded), `disabled /collapsed` (sidebar collapsed), agent variant (extra “Queue” item). Entrance animation only when `enteredFromAuth`.

Dependencies

- Children: `UserProfileSidebar`, `ShellTopbar`, `FloatingChatWidget`, `AnimatedOutlet`.
- Libraries: `framer-motion`, `react-router-dom`, `lucide-react`.
- Helpers: `getDisplayname`, `getUserAvatarUrl`, `hasAuthTransition`.

Accessibility

Sidebar `aria-label="Primary"`; mobile nav `aria-label="Primary mobile"`; the collapse toggle and links carry labels (in `UserProfileSidebar`).

Usage example

```
<Route element={<ProtectedRoute><CurrencyProvider><AppShell /></CurrencyProvider></ProtectedRoute>>
  <Route path="/dashboard" element={<DashboardPage />} />
</Route>
```

Related / used by

The layout route for all protected pages. Renders the nav, topbar, and chat widget.

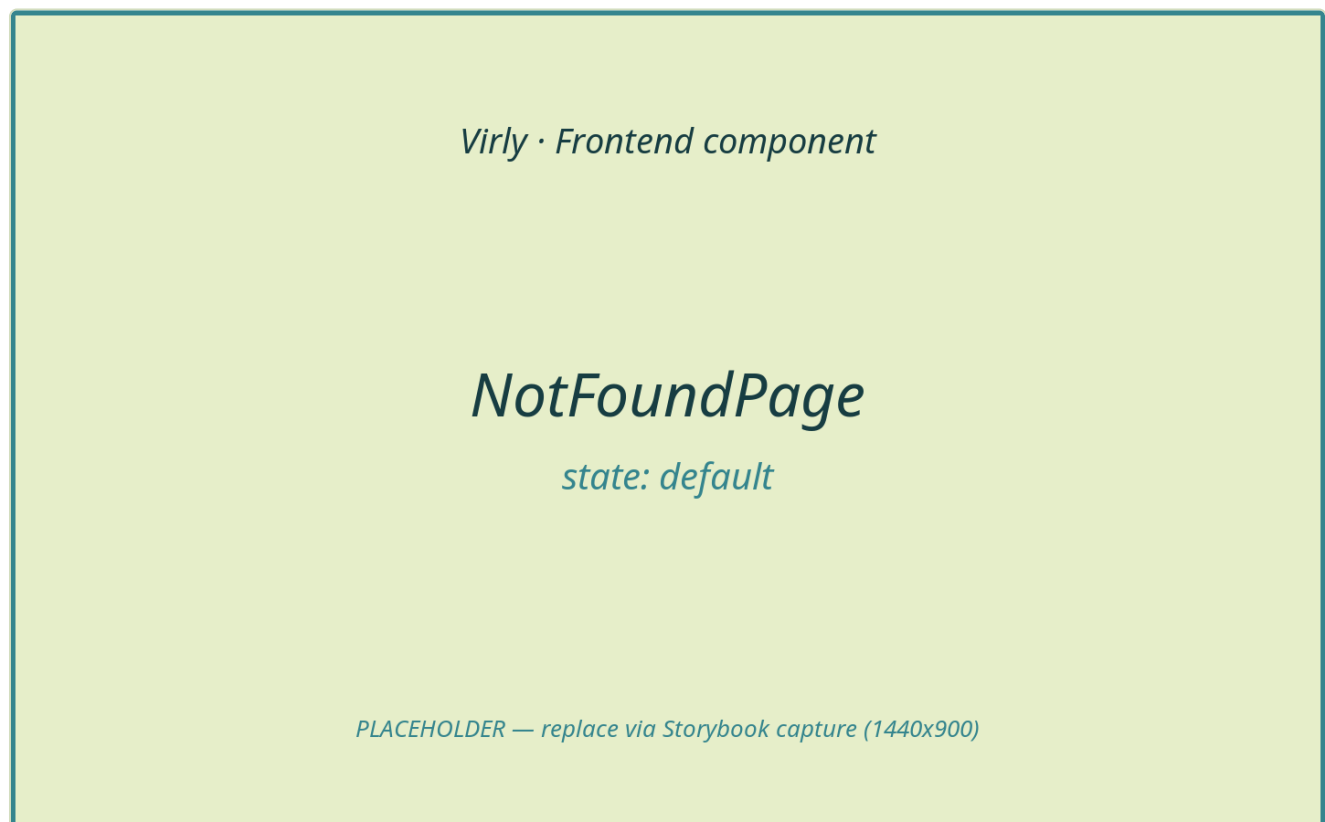
Notes / gotchas

`AnimatedOutlet` freezes the outlet captured at mount so the exiting route keeps rendering during the `AnimatePresence` exit. The “Queue” nav item appears only for `support_agent` / `sales_agent` / `support_manager` / `admin` roles.

NotFoundPage

- **Path:** `client/src/features/not-found/NotFoundPage.tsx`
- **Category:** page | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** The 404 page: a “declined receipt” slip plus navigation actions.

Screenshot(s)



404 slip with “Back to dashboard” / “Go back” actions.

Purpose & context

The catch-all route. It derives a stable, deterministic “reference” from the requested path and a print timestamp, then renders `NotFoundSlip` with delayed navigation actions.

Anatomy

`nf-screen` → `nf-stage` → `NotFoundSlip` + animated actions (`Link` to `/`, “Go back” button calling `navigate(-1)`).

Props / API

None.

State & data

- `useMemo` derives `printedAt` + `reference` from the requested path.
- Hooks: `useLocation`, `useNavigate`.

Interactions & events

- “Back to dashboard” → `/`.
- “Go back” → `navigate(-1)`.

States & variants

- `default` only. Loading/error/success/disabled: N/A.

Dependencies

- Children: `NotFoundSlip`, `Link`, `lucide-react`.
- Libraries: `framer-motion`.

Accessibility

`aria-label="Page not found, error 404"` on the main; actions are real links / buttons.

Usage example

```
<Route path="*" element={<NotFoundPage />} />
```

Related / used by

Routed by `App` (catch-all). Wraps `NotFoundSlip`.

Notes / gotchas

The reference number is a deterministic hash of the path, so the same dead link always “declines” with the same reference.

NotFoundSlip

- **Path:** `client/src/components/NotFoundSlip.tsx`
- **Category:** feature | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** The presentational 404 surface rendered as a declined bank receipt (printed, tilted, stamped “Declined / No Such Route”).

Screenshot(s)



The declined receipt with request ledger + barcode.

Purpose & context

A self-contained animated receipt used by `NotFoundPage`. It prints top-to-bottom, settles at a tilt, slams down a “Declined” stamp, and lists the failed request as ledger rows.

Anatomy

Merchant header, “404” headline + stamp, ledger rows (status, requested, method, posted, reference), totals (“Pages found 0”, “Balance of luck \$0.00”), barcode, footer.

Props / API

Prop	Type	Required	Default	Description
<code>requestId</code>	<code>string</code>	Yes	—	The dead path shown on the “Requested” line.

Prop	Type	Required	Default	Description
<code>printedAt</code>	<code>string</code>	Yes	—	Pre-formatted print timestamp.
<code>reference</code>	<code>string</code>	Yes	—	Deterministic “declined” reference number.

State & data

None (deterministic barcode computed at module load).

Interactions & events

`whileHover` straightens the slip slightly. No callbacks (actions live in `NotFoundPage`).

States & variants

- `default` only. Loading/empty/error/success/disabled: N/A.

Dependencies

- Libraries: `framer-motion`.

Accessibility

Decorative paper/stamp/barcode are `aria-hidden`; the host page provides the `aria-label`. Content is visual text.

Usage example

```
<NotFoundSlip requested={requested} printedAt={printedAt} reference={reference} />
```

Related / used by

Rendered by `NotFoundPage`.

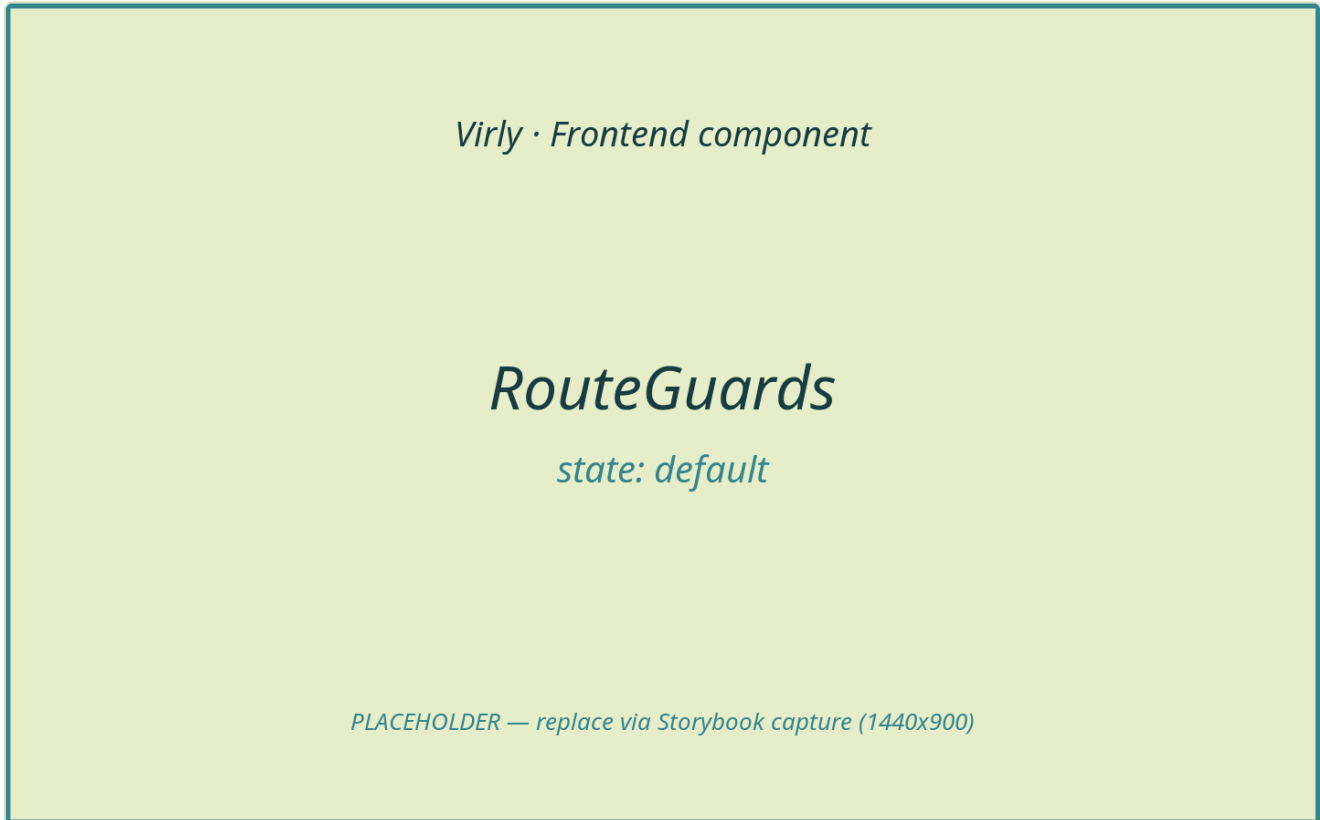
Notes / gotchas

Pure presentation — pair it with navigation actions in the host page (it ships none of its own).

RouteGuards

- **Path:** `client/src/components/RouteGuards.tsx`
- **Category:** provider/container | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** `ProtectedRoute` and `GuestRoute` — the auth-aware redirect wrappers that gate the app shell vs the auth pages.

Screenshot(s)



No own UI; this represents a guard resolving to its child or a redirect.

Purpose & context

Two small wrappers used in the route table. `ProtectedRoute` renders children only when authenticated (else redirects to `/login`, preserving `from`). `GuestRoute` renders children only when unauthenticated (else redirects to `/dashboard`, with a special case that keeps `/login` rendered for personal-details / auth-transition continuations). Both render `null` while auth is still loading.

Anatomy

- `ProtectedRoute({ children })` — loading → `null`; unauth → `Navigate /login`; else children.
- `GuestRoute({ children })` — loading → `null`; auth → `Navigate /dashboard` (with the login/needs-details exception); else children.

Props / API

Component	Prop	Type	Required	Default	Description
<code>ProtectedRoute</code>	<code>children</code>	<code>ReactNode</code>	Yes	—	Protected subtree.
<code>GuestRoute</code>	<code>children</code>	<code>ReactNode</code>	Yes	—	Guest-only subtree.

State & data

- Hooks: `useAuth` , `useLocation` . No data fetching.

Interactions & events

Redirect via `<Navigate>`; `ProtectedRoute` passes `state={{ from: location }}` so login can return the user to the originally-requested route.

States & variants

- `loading (null)` , `authenticated` , `unauthenticated` . Empty/error/success/ disabled: N/A.

Dependencies

- Libraries: `react-router-dom` .
- Helper: `hasAuthTransition` , `useAuth` .

Accessibility

N/A (no DOM of their own).

Usage example

```
<Route path="/login" element={<GuestRoute><LoginPage /></GuestRoute>} />
<Route element={<ProtectedRoute><CurrencyProvider><AppShell /></CurrencyProvider></ProtectedRoute>} />
```

Related / used by

Used throughout `App` 's route table.

Notes / gotchas

Returning `null` while `isLoading` prevents a flash of the wrong surface before the session resolves.

ShellTopbar

- **Path:** `client/src/components/ShellTopbar.tsx`
- **Category:** layout | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** The top bar: animated wordmark, currency selector, and the user's name + balance + avatar.

Screenshot(s)

Virly · Frontend component

ShellTopbar

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Wordmark, currency selector, user meta + balance.

Purpose & context

Rendered by `AppShell` at the top of the content column. It shows the brand link home, the global `CurrencySelector`, and the user's display name + balance (formatted in the selected display currency).

Anatomy

Brand `Link` (`AnimatedText` “Virly”) + `topbar-actions` (`CurrencySelector`, user meta with name + balance, initials avatar).

Props / API

Prop	Type	Required	Default	Description
<code>displayName</code>	<code>string</code>	Yes	—	User display name.
<code>email</code>	<code>string</code>	Yes	—	Used to derive the initials avatar.
<code>balance</code>	<code>number</code>	Yes	—	Authoritative ILS balance to format.
<code>enteredFromAuth</code>	<code>boolean</code>	Yes	—	Enables the slide-down entrance animation.

State & data

- `useCurrency().formatAmount` for the balance. No data fetching.

Interactions & events

- Wordmark → `/dashboard`.
- Currency change handled by `CurrencySelector / CurrencyProvider`.

States & variants

- `default`. Entrance animation only when `enteredFromAuth`. Loading/empty/ error/success/ disabled: N/A.

Dependencies

- Children: `CurrencySelector`, `AnimatedText`, `Link`.
- Libraries: `framer-motion`.

Accessibility

Brand link `aria-label="Virly home"`; balance has an `sr-only` "Current balance" prefix; avatar `aria-hidden`.

Usage example

```
<ShellTopbar displayName={displayName} email={auth.user?.email ?? ""}  
  balance={auth.user?.balance ?? 0} enteredFromAuth={enteredFromAuth} />
```

Related / used by

Rendered by `AppShell`. Hosts `CurrencySelector`.

Notes / gotchas

The balance comes from `auth.user.balance`, which is kept in sync with the server by `updateBalance` after transfers — the topbar never computes it.

UserProfileSidebar

- **Path:** `client/src/components/ui/menu.tsx`
- **Category:** layout | **Feature area:** Layout / Navigation | **Tier:** Full
- **Summary:** The primary sidebar: user identity, collapsible nav items (with separators and bottom-pinned items), and a logout button.

Screenshot(s)

Virly · Frontend component

UserProfileSidebar

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Expanded sidebar with nav links + logout.

Virly · Frontend component

UserProfileSidebar

state: disabled

PLACEHOLDER — replace via Storybook capture (1440x900)

Collapsed sidebar (icons only).

Purpose & context

The desktop navigation. It renders the user header (avatar, name, auto-fit email), a collapse toggle, the nav list (separators + bottom-pinned items like Settings), and a logout action. All routing/state is provided by `AppShell`.

Anatomy

- User block: avatar, name, fit-to-width email, collapse toggle.
- Divider.
- Nav: main items + spacer + bottom-pinned items (`NavLink` s with chevrons, optional separators).
- Footer: logout button.

Props / API

Prop	Type	Required	Default	Description
<code>user</code>	<code>UserProfile</code> ({ name, email, avatarUrl })	Yes	—	Sidebar identity block.
<code>navItems</code>	<code>NavItem[]</code>	Yes	—	{ icon, label, href, isSeparator?, pinToBottom? }.
<code>logoutItem</code>	{ icon, label, onClick }	Yes	—	The logout action.
<code>collapsed</code>	<code>boolean</code>	No	<code>false</code>	Collapsed (icon-only) state.
<code>onToggleCollapse</code>	<code>() => void</code>	No	—	Collapse toggle handler.
<code>className</code>	<code>string</code>	No	—	Extra classes (merged via <code>cn</code>).

State & data

- `useFitText` (custom `useLayoutEffect` hook) shrinks the email to fit. No data fetching.

Interactions & events

- Nav `NavLink` s → routes.
- Collapse toggle → `onToggleCollapse`.
- Logout → `logoutItem.onClick`.

States & variants

- `default` (expanded), `disabled /collapsed`. Loading/empty/error/success: N/A.

Dependencies

- Libraries: `framer-motion`, `react-router-dom`, `lucide-react`.
- Helper: `cn`, `useFitText`.

Accessibility

`aria-label="User profile menu"`; collapse toggle has `aria-expanded` + `aria-controls` + `aria-label`; nav region has `role="navigation"`; links carry `aria-label` and (collapsed) `title`.

Usage example

```
<UserProfileSidebar
  user={{ name, email, avatarUrl }}
  navItems={items}
  logoutItem={{ label: "Log out", icon: <Logout />, onClick: handleLogout }}
  collapsed={isSidebarCollapsed}
  onToggleCollapse={toggleSidebar}
/>
```

Related / used by

Rendered by `AppShell` (forwarded ref).

Notes / gotchas

`useFitText` observes the parent's size and rescales the email font down to a floor of 8px, so long emails never overflow the collapsed/expanded rail.

Shared UI

This area holds reusable primitives (the hand-rolled `Primitives` toolkit and the shadcn-style `components/ui/*` set), the currency context + selector, and the decorative background. It also hosts three feature surfaces that genuinely do not fit any of the seven fixed areas — `SettingsPage` and the video-session pages — placed here per the documentation rules with an explicit note (flagged for human review in the inventory). Video sessions are **human-agent (Jitsi) calls**, distinct from the AI Assistant; the assistant only links into them via its `video_session_cta` block.

Most entries are **Lite** (primitives). The `Primitives.tsx` file exports multiple primitives and is documented as a single grouped entry. Screenshots are placeholders pending Storybook capture.

Components in this area

- `AgentVideoSessionsPage` (*fits none — video*)
- `AnimatedText`
- `Avatar`
- `BentoCard`
- `Button` (shadcn/ui)
- `CurrencyProvider`
- `CurrencySelector`
- `JitsiMeeting` (*fits none — video*)
- `OrderConfirmationCard`
- `Primitives` (UI toolkit)
- `Select` (shadcn/ui)
- `SettingsPage` (*fits none*)
- `ShaderBackground`
- `VideoSessionPage` (*fits none — video*)

AgentVideoSessionsPage

- **Path:** `client/src/features/video/AgentVideoSessionsPage.tsx`
- **Category:** page | **Feature area:** Shared UI (does not fit the seven fixed areas) | **Tier:** Full
- **Summary:** Internal agent queue for video sessions: filter, assign, join, and end support/sales calls. (*No matching fixed area; placed here per the rules.*)

Screenshot(s)

Virly · Frontend component

AgentVideoSessionsPage

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Queue list + join stage for agents.

Virly · Frontend component

AgentVideoSessionsPage

state: empty

PLACEHOLDER — replace via Storybook capture (1440x900)

No sessions match the filters.

Virly · Frontend component

AgentVideoSessionsPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Skeleton while sessions load.

Virly · Frontend component

AgentVideoSessionsPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Non-agent access denied / load error.

Purpose & context

A role-gated internal tool (support/sales agents, support managers, admins) for working the video queue. It lists sessions filtered by type/status, lets an agent assign and join (opening a `JitsiMeeting`), and end the active session. Customer financial details are intentionally **not** shown here.

Anatomy

- Access-denied branch for non-agents (`ShieldAlert`).
- `PageHeader` + Refresh; type/status filter card; session list (assign/join per row); agent stage card hosting `JitsiMeeting` .

Props / API

None. (Reads `auth.user.role` .)

State & data

- Local state: `sessions` , `status` , `type` , `activeSession` , `jitsi` , `isLoading` , `isBusy` , `error` .
- Hooks: `useAuth` , `useCallback` , `useEffect` , `useMemo` , `useState` .
- Data: `api.adminVideoSessions` (GET `/api/admin/video-sessions`), `api.assignVideoSession` (POST `.../:id/assign`), `api.adminVideoJoinToken` (POST `.../:id/join-token`), `api.adminEndVideoSession` (POST `.../:id/end`).

Interactions & events

Refresh → reload; Assign → assign + reload; Join → join token + open meeting; End → end + reload. Type filter is locked for single-type roles.

States & variants

- `default` , `loading` , `empty` , `error` , `access-denied` , `active-meeting` . Success/disabled: buttons disable while busy or for terminal statuses.

Dependencies

- Children: `JitsiMeeting` , `Primitives` (`Button` , `Card` , `ErrorBanner` , `PageHeader` , `PageStack` , `ResponsiveGrid` , `Skeleton`), `lucide-react` .

Accessibility

`PageHeader` heading; status pills are text; filters are labelled selects.

Usage example

```
<Route path="/agent/video-sessions" element={<Suspense fallback={<RouteFallback />}><AgentVideoSessionsPage /></Suspense>} />
```

Related / used by

Lazy-routed by `App` ; reachable via the “Queue” sidebar item (agent roles). Pairs with `VideoSessionPage` (customer side) and `JitsiMeeting` .

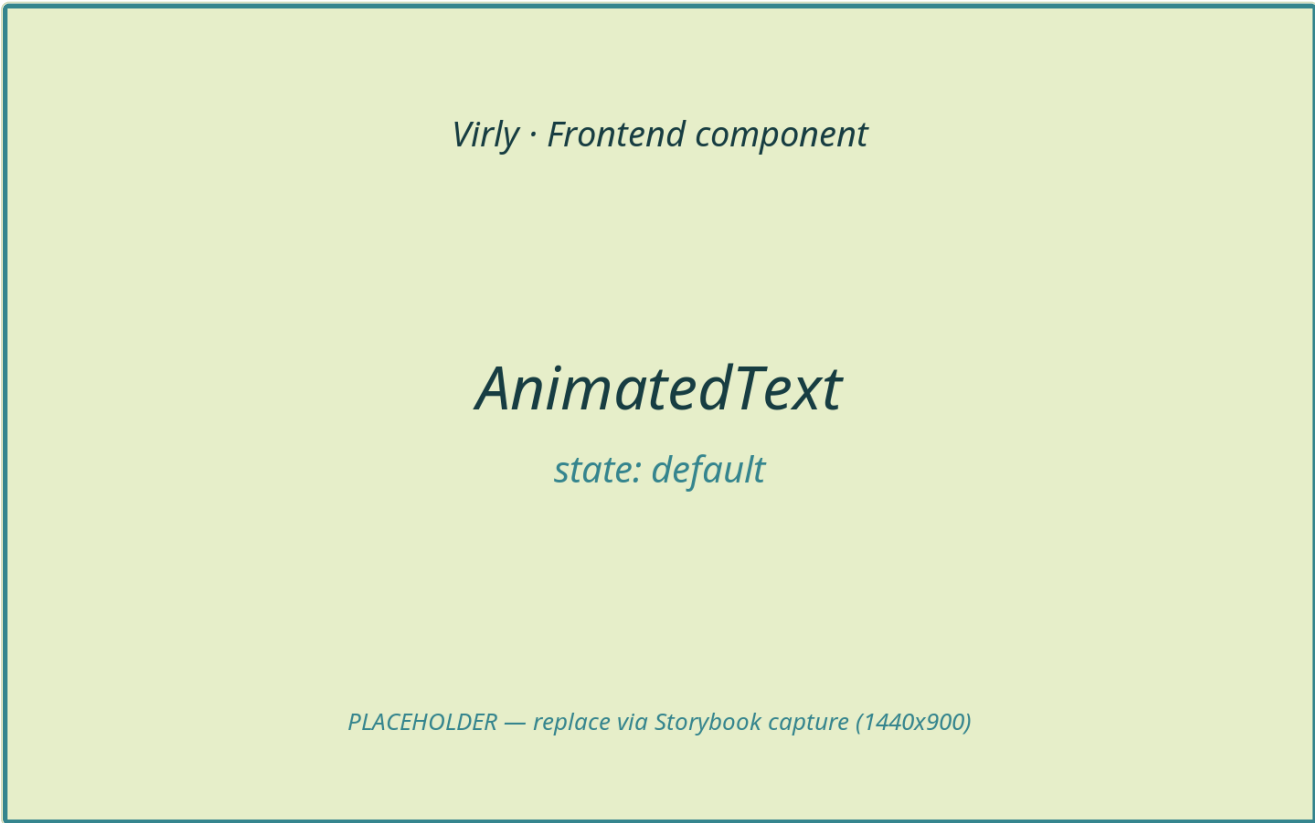
Notes / gotchas

The “Queue” nav entry and this page are both gated to agent roles; the page also re-checks the role and renders an access-denied card defensively.

AnimatedText

- **Path:** `client/src/components/ui/animated-text.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** Letter-by-letter reveal with an animated underline, used for the brand wordmark.

Screenshot(s)



“Virly” revealing letter by letter with an underline sweep.

Props / API

Prop	Type	Required	Default	Description
<code>text</code>	<code>string</code>	Yes	—	Text to animate.
<code>duration</code>	<code>number</code>	No	<code>0.19</code>	Per-letter stagger.

Prop	Type	Required	Default	Description
<code>delay</code>	<code>number</code>	No	<code>0.18</code>	Delay before children.
<code>replay</code>	<code>boolean</code>	No	<code>true</code>	Whether to animate to visible.
<code>as</code>	<code>"h1"..."h6"</code> <code>\ "p" \ </code> <code>"span"</code>	No	<code>"h1"</code>	Rendered element.
<code>className</code> / <code>textClassName</code> / <code>underlineClassName</code>	<code>string</code>	No	—	Style hooks.
<code>underlineGradient</code> / <code>underlineHeight</code> / <code>underlineOffset</code>	<code>string</code>	No	—	Underline style classes.

States & variants

`default` (visible/animating). When `replay=false` it stays hidden. Honors reduced-motion via the `app MotionConfig`. Loading/empty/error/disabled: N/A.

Usage example

```
<AnimatedText text="Virly" as="span" duration={0.07} textClassName="topbar-wordmark-text" />
```

Avatar

- **Path:** `client/src/components/ui/avatar.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** Radix-based avatar set (`Avatar` , `AvatarImage` , `AvatarFallback`).

Screenshot(s)

Virly · Frontend component

Avatar

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Image avatar with an initials fallback.

Props / API

Thin wrappers over `@radix-ui/react-avatar` primitives; all accept the underlying Radix props plus `className` (merged via `cn`). No custom props.

Export	Wraps	Notes
<code>Avatar</code>	<code>AvatarPrimitive.Root</code>	Round, fixed <code>h-10 w-10</code> by default.
<code>AvatarImage</code>	<code>AvatarPrimitive.Image</code>	<code>object-cover</code> .
<code>AvatarFallback</code>	<code>AvatarPrimitive.Fallback</code>	Shown until the image loads.

States & variants

`default` (image), `fallback` (no/failed image). Loading is handled by Radix.

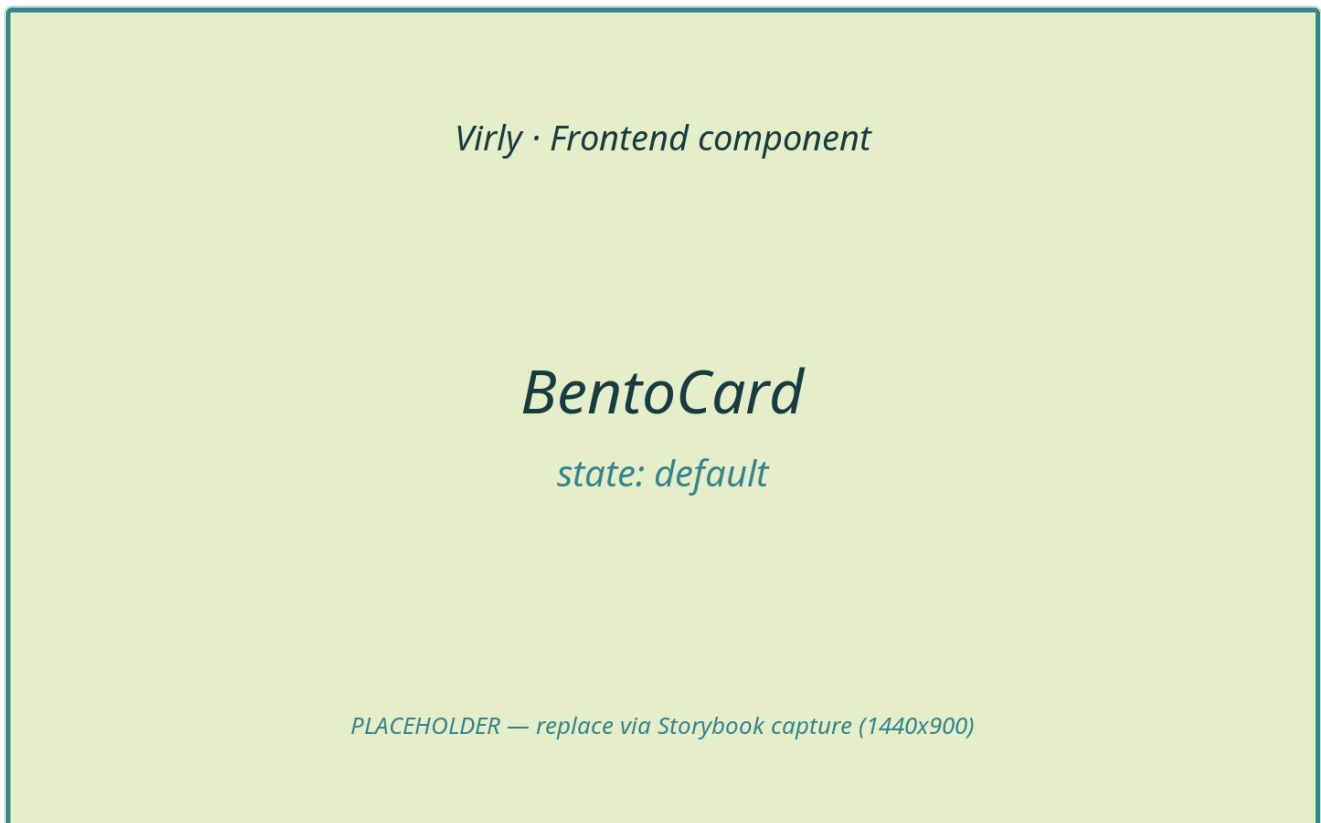
Usage example

```
<Avatar><AvatarImage src={url} alt="" /><AvatarFallback>AI</AvatarFallback></Avatar>
```

BentoCard

- **Path:** `client/src/components/ui/bento-card.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** A self-contained, tabbed “project dashboard” showcase card (decorative/sample content).

Screenshot(s)



Tabbed bento showcase (Dashboard / Management / Threads / Resources).

Props / API

None. (Internal `activeTab` state; sample data hard-coded.)

States & variants

`default` plus the four tab variants. Loading/empty/error/success/disabled: N/A.

Usage example

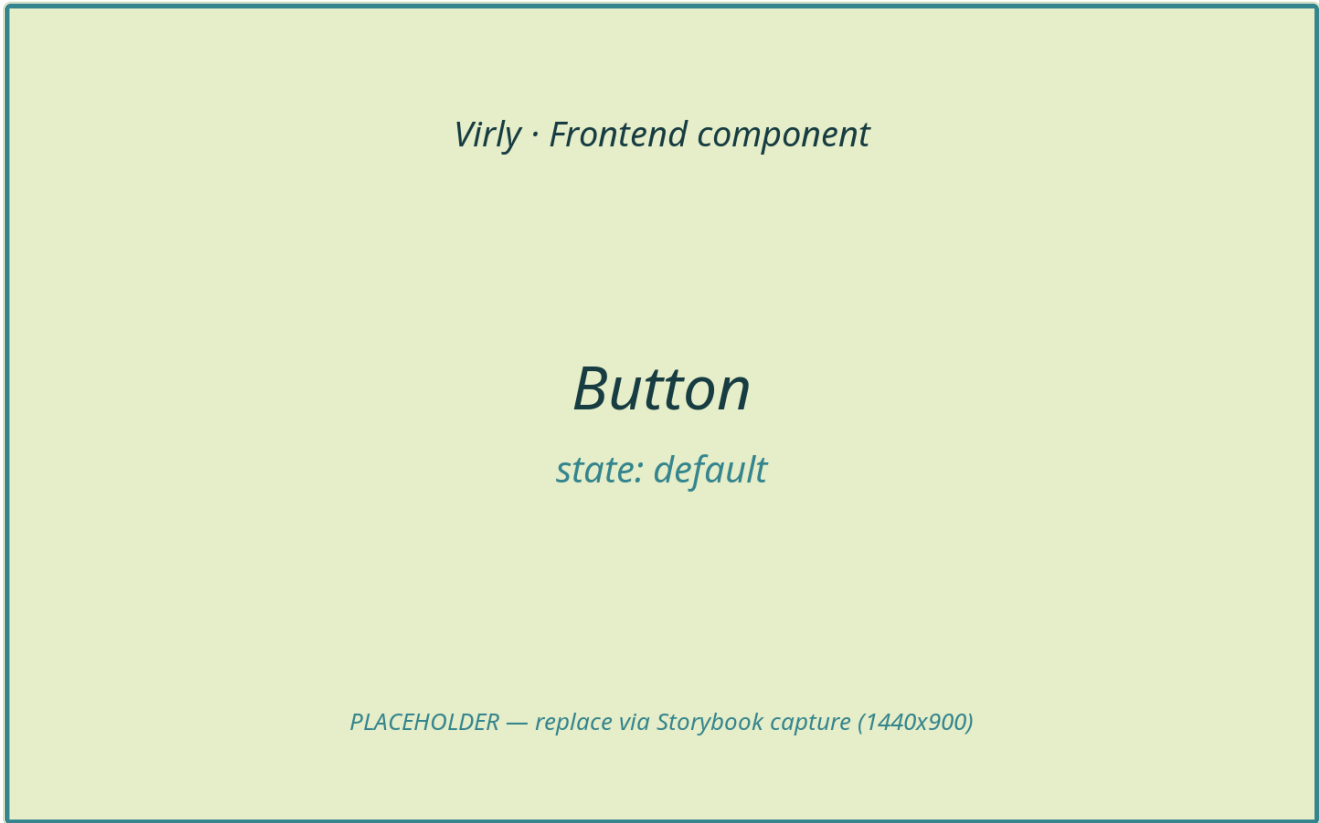
```
<BentoCard />
```

Note: This appears to be a showcase/sample component (Hugeicons + hard-coded demo content) and is not part of the money flows. Verify whether it is still referenced before relying on it.

Button (shadcn/ui)

- **Path:** `client/src/components/ui/button.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** The shadcn/CVA button used by the chat widget and shadcn-style cards (distinct from the `Primitives Button`).

Screenshot(s)



Variant/size matrix (default, secondary, outline, ghost, destructive, link).

Props / API

Prop	Type	Required	Default	Description
variant	<code>"default"</code> <code>\\</code> <code>"destructive"</code> <code>\\</code> <code>"outline"</code> <code>\\</code> <code>"secondary"</code> <code>\\</code> <code>"ghost"</code> <code>\\</code> <code>"link"</code>	No	<code>"default"</code>	CVA variant.

Prop	Type	Required	Default	Description
size	"default" \\ "sm" \\ "lg" \\ "icon"	No	"default"	CVA size.
asChild	boolean	No	false	Render via Radix Slot .
... ButtonHTM LAttribut es	—	—	—	Standard button attributes.

States & variants

All CVA `variant` / `size` combinations; `disabled` lowers opacity + blocks pointer events. Loading/empty/error/success: caller-driven.

Usage example

```
<Button variant="ghost" size="icon" aria-label="Close chat"><X /></Button>
```

Note: Two `Button`s exist. This shadcn/CVA one (Tailwind) is used by `FloatingChatWidget` / `OrderConfirmationCard`; the `Primitives Button` (classic CSS) is used by pages/forms. Pick by surrounding style system.

CurrencyProvider

- **Path:** `client/src/features/currency/CurrencyProvider.tsx`
- **Category:** provider/context | **Feature area:** Shared UI | **Tier:** Full
- **Summary:** Context that holds the selected display currency + daily exchange rates and exposes an ILS-aware `formatAmount` .

Screenshot(s)

Virly · Frontend component

CurrencyProvider

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

No own UI; represents currency-formatted amounts across the shell.

Purpose & context

Wraps the protected app branch. It reads the stored display currency, fetches the daily rates (`api.exchangeRates`), and exposes `formatAmount(amountILs)` plus `setCurrency`. When rates are unavailable, amounts degrade gracefully to plain ILS. Ledger values remain authoritative ILS underneath; this only affects display.

Anatomy

`CurrencyContext` (+ `useCurrency`) exposing `currency`, `setCurrency`, `rates`, `conversionAvailable`, `formatAmount`. Accepts test/seed overrides.

Props / API

Prop	Type	Required	Default	Description
<code>children</code>	<code>ReactNode</code>	Yes	—	Subtree consuming currency.
<code>initialCurrency</code>	<code>DisplayCurrency</code>	No	—	Test/seed override (else reads storage).
<code>initialRates</code>	<code>ExchangeRatesResponse</code> <code>null</code>	No	<code>null</code>	Test/seed override (skips fetch).

Context value: `currency`, `setCurrency(c)`, `rates`, `conversionAvailable`, `formatAmount(amountIls) => string`.

State & data

- Local state: `currency`, `rates`.
- Data: `api.exchangeRates()` → `GET /api/exchange-rates/current` (skipped when `initialRates` is provided).
- Persistence: `storeCurrency` / `readStoredCurrency` (`localStorage`).

Interactions & events

`setCurrency` updates state + persists; `formatAmount` reformats based on `currency` + `rates`.

States & variants

- `default` (rates loaded), no-rates fallback (ILS only, `conversionAvailable=false`). Loading/empty/error/disabled: N/A (no own UI).

Dependencies

- Helpers: `formatIlsAmount`, `readStoredCurrency`, `storeCurrency`.

Accessibility

N/A (no DOM). Consumers render the amounts.

Usage example

```
<CurrencyProvider><AppShell /></CurrencyProvider>
// const { formatAmount } = useCurrency();
```

Related / used by

Wraps the protected branch in `App`. Consumed by `ShellTopbar`, `TransferPage`, `AccountStatement`, `TransactionList`, `TransactionReceipt`, `RelationshipSummaryCard`, `SettingsPage`, `CurrencySelector`, etc.

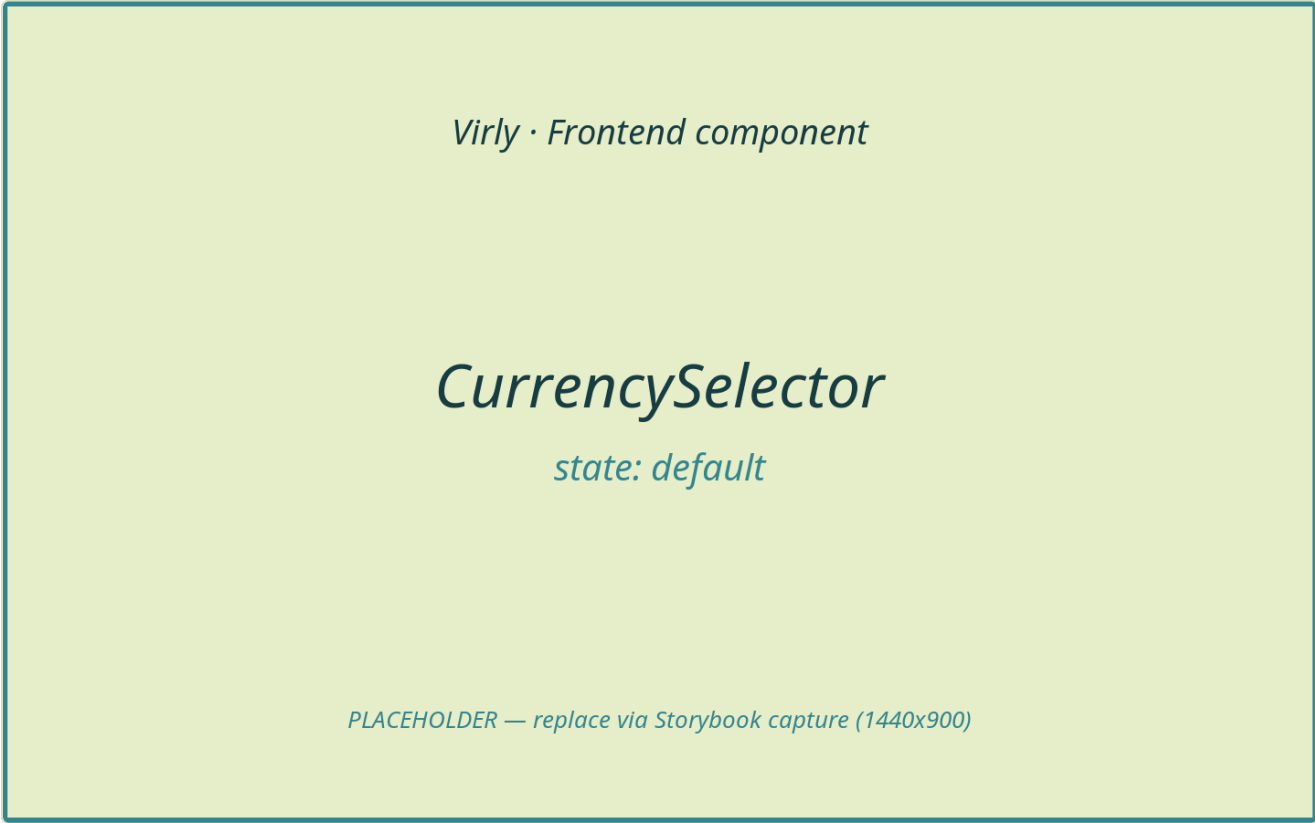
Notes / gotchas

The default context value formats in ILS, so `useCurrency` is safe even outside the provider (degrades to ILS) — though in practice it is always provided inside the shell.

CurrencySelector

- **Path:** `client/src/features/currency/CurrencySelector.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** The display-currency dropdown (ILS / USD / EUR) for the topbar.

Screenshot(s)



Native select bound to the currency context.

Props / API

Prop	Type	Required	Default	Description
currency	DisplayCurrency	No	context value	Controlled value override.
onCurrencyChange	(currency : DisplayCurrency) => void	No	context setCurrency	Change handler override.

States & variants

default . Binds to useCurrency by default; both props are optional overrides. Loading/empty/error/success/disabled: N/A.

Usage example

```
<CurrencySelector />
```

JitsiMeeting

- **Path:** `client/src/features/video/JitsiMeeting.tsx`
- **Category:** feature | **Feature area:** Shared UI (does not fit the seven fixed areas) | **Tier:** Full
- **Summary:** Mounts a Jitsi external-API meeting into a container and bridges its join/leave/error events. *(No matching fixed area; placed here per the rules.)*

Screenshot(s)



Embedded Jitsi meeting frame.

Virly · Frontend component

JitsiMeeting

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

"Preparing secure video..." while the runtime loads.

Virly · Frontend component

JitsiMeeting

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Runtime failed to load (reported via `onError`).

Purpose & context

A thin wrapper around the Jitsi `external_api.js` runtime. It lazily injects the script for the configured domain, instantiates the meeting with the server-issued join config (room, JWT, overrides), and forwards lifecycle events.

Anatomy

A loading overlay + a container `div` that Jitsi renders into. Script loading is deduplicated per domain.

Props / API

Prop	Type	Required	Default	Description
<code>jitsi</code>	<code>JitsiJoinConfig</code>	Yes	—	Server-issued join config (domain, room, JWT, overrides).
<code>displayName</code>	<code>string</code>	Yes	—	Name shown in the meeting.
<code>onJoined</code>	<code>() => void</code>	No	—	Fired on <code>videoConferenceJoined</code> .
<code>onLeft</code>	<code>() => void</code>	No	—	Fired on leave / <code>readyToClose</code> .
<code>onError</code>	<code>(message: string) => void</code>	No	—	Script/init failure message.

State & data

- Local state: `loading`; refs for the container + API instance.
- No app API calls; loads the external Jitsi script.

Interactions & events

- Mount → load script → instantiate → add listeners.
- Unmount → `dispose()` the API instance.

States & variants

- `default` (joined), `loading`, `error` (via `onError`). Empty/success/disabled: N/A.

Dependencies

- External: Jitsi `external_api.js` (`window.JitsiMeetExternalAPI`).

Accessibility

Relies on the embedded Jitsi UI for in-call a11y; the wrapper shows a visible loading message.

Usage example

```
<JitsiMeeting jitsi={jitsi} displayName={displayName} onError={setError}  
onLeft={endSession} />
```

Related / used by

Rendered by `VideoSessionPage` (customer) and `AgentVideoSessionsPage` (agent).

Notes / gotchas

The join config (including any JWT) is issued by the backend per session; the component never mints credentials itself.

OrderConfirmationCard

- **Path:** `client/src/components/ui/order-confirmation-card.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** A generic success/confirmation card (icon, title, detail rows, CTA) in the shadcn style.

Screenshot(s)



Transaction-completed card with detail rows + a CTA.

Props / API

Prop	Type	Required	Default	Description
orderId	string	Yes	—	Shown as “Transaction ID”.
paymentMethod	string	Yes	—	Shown as “Counterparty”.
dateTime	string	Yes	—	Shown as “Date & Time”.
totalAmount	string	Yes	—	Total (debit if it starts with -).
onGoToAccount	() => void	Yes	—	CTA handler.
reason	string \ null	No	—	Optional reason row.
title	string	No	"Transaction completed successfully"	Heading.
buttonText	string	No	"Close"	CTA label.
icon	ReactNode	No	CheckCircle2	Header icon.
className	string	No	—	Extra classes.

States & variants

default ; debit vs credit styling on the total. aria-live="polite" . Loading/empty/error/disabled: N/A.

Usage example

```
<OrderConfirmationCard orderId={id} paymentMethod={email} dateTime={when}
totalAmount={amount} onGoToAccount={close} />
```

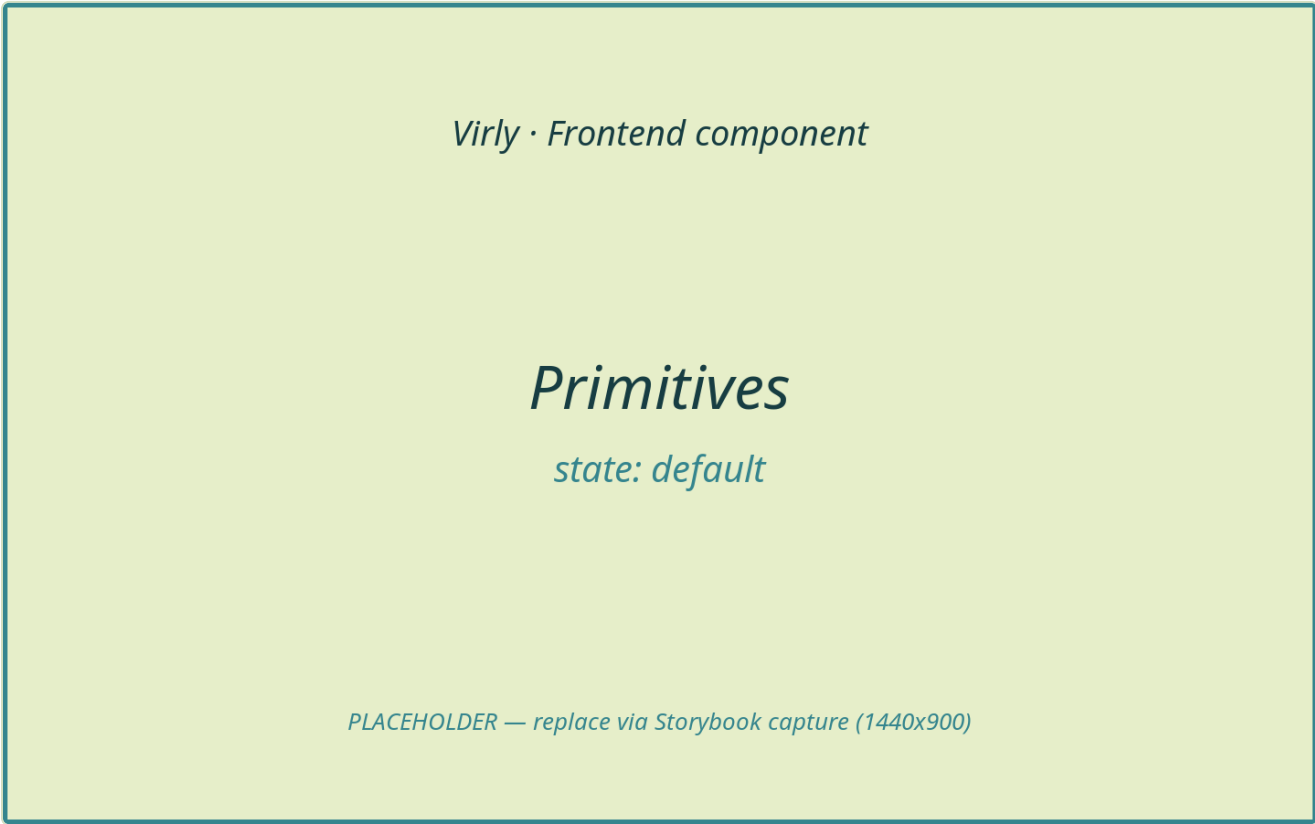
Note: A generic, presentational confirmation card. The live transfer/AI flows use TransferCheque (success) and the assistant confirmation card; verify where this one is wired before relying on it.

Primitives (UI toolkit)

- **Path:** client/src/components/Primitives.tsx
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite

- **Summary:** The hand-rolled, CSS-class-based building blocks used across the pages and forms (distinct from the shadcn `components/ui/*` set).

Screenshot(s)



Gallery: buttons, fields, banners, empty state, skeleton.

Exports (each a small primitive)

Export	Key props	Notes
Button	variant (primary secondary ghost danger), ...button attrs	Classic CSS button.
Card	children , className	section.card wrapper.
PageStack	children , className	Vertical page stack.
ResponsiveGrid	variant (sidebar dashboard split filters), children	Layout grid.
Field	label , error , hint , ...input attrs	Labelled input + error/hint.
TextareaField	label , name , value , onChange , error , hint , maxLength	Labelled textarea.
PageHeader	eyebrow , title , children (actions)	Page heading + actions slot.
ErrorBanner	message	role="alert" banner.

Export	Key props	Notes
SuccessBanner	message	role="status" banner.
EmptyState	title, message, icon, children	Empty placeholder + actions.
Skeleton	rows (default 3)	"Printing..." loading placeholder.

States & variants

- Button : variants + disabled.
- Field / TextareaField : default, error (aria-invalid), hint.
- EmptyState : empty placeholder (used as the empty state elsewhere).
- Skeleton : loading (aria-busy).

Accessibility

Field / TextareaField label via htmlFor / id and set aria-invalid ; banners carry role ; Skeleton is role="status" aria-busy . TODO: link field errors via aria-describedby .

Usage example

```
<PageStack>
  <PageHeader eyebrow="" title="Transactions" />
  <Card><Field label="Email" name="email" error={err} onChange={...} /></Card>
  {isLoading ? <Skeleton rows={6} /> : <List />}
</PageStack>
```

Related / used by

Used by nearly every page/form in the app (Auth, Dashboard, Transfers, Transactions, Settings, Video).

Note: Documented as one grouped entry because the file exports many small primitives. The shadcn Button (ui/button.tsx) is separate — see above.

Select (shadcn/ui)

- **Path:** client/src/components/ui/select.tsx
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** The Radix-based select set (Select , SelectTrigger , SelectContent , SelectItem , ...) used for the chat agent picker.

Screenshot(s)

Virly · Frontend component

Select

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Open select with items + check indicator.

Props / API

Re-exports of `@radix-ui/react-select` parts, styled via `cn`. No custom props beyond the Radix component props (+ `className`). Exports: `Select`, `SelectGroup`, `SelectValue`, `SelectTrigger`, `SelectContent`, `SelectLabel`, `SelectItem`, `SelectSeparator`, `SelectScrollUpButton`, `SelectScrollDownButton`.

States & variants

`default`, `open/closed` (Radix data-state animations), `disabled`. Loading/empty/ error/success: N/A.

Usage example

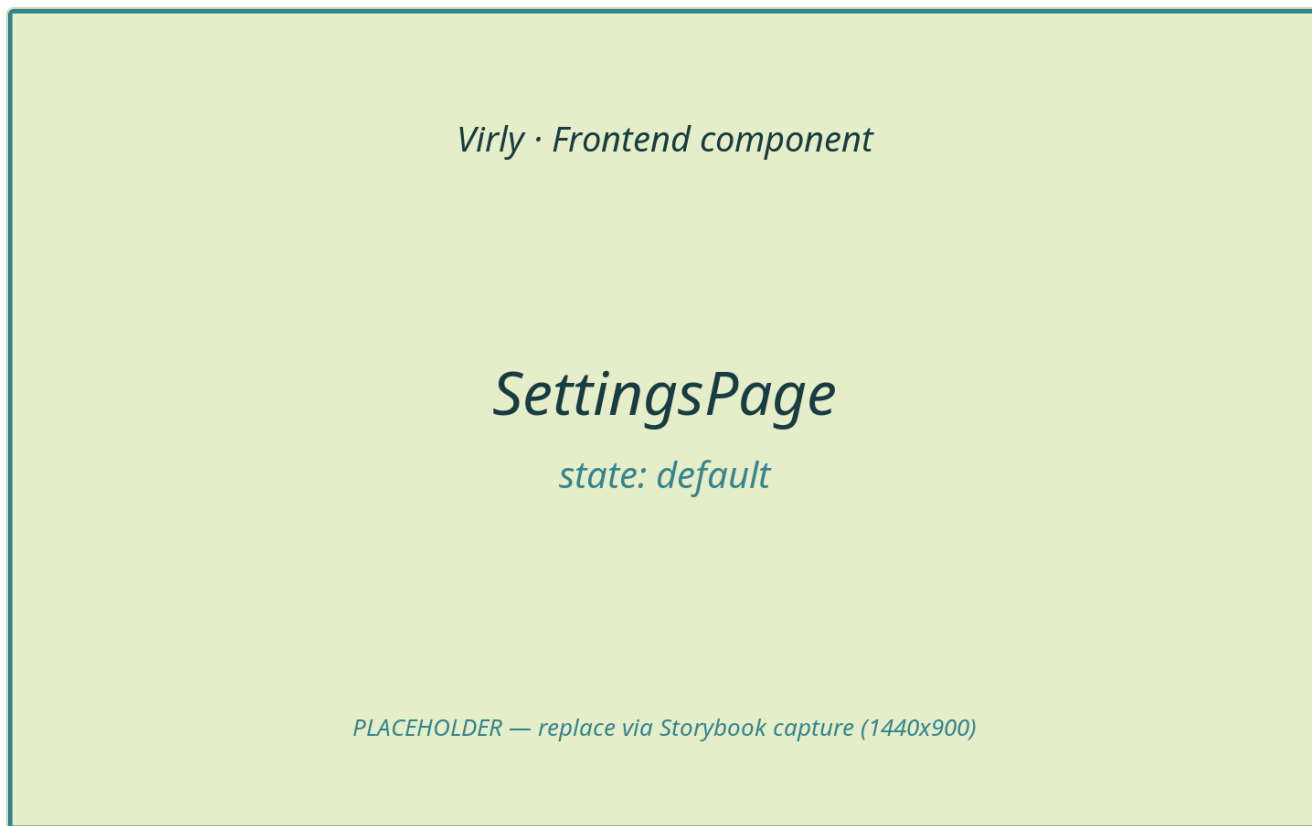
```
<Select value={agent} onValueChange={setAgent}>
  <SelectTrigger aria-label="Choose an assistant">...</SelectTrigger>
  <SelectContent><SelectItem value="oshri">Oshri</SelectItem></SelectContent>
</Select>
```

SettingsPage

- **Path:** `client/src/features/settings/SettingsPage.tsx`
- **Category:** page | **Feature area:** Shared UI (does not fit the seven fixed areas) | **Tier:** Full

- **Summary:** Account settings: view/edit personal details, see account email + balance, and sign out. (No matching fixed area; placed here per the rules.)

Screenshot(s)



Personal details (read mode) + account + session cards.

Virly · Frontend component

SettingsPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Skeleton while details load.

Virly · Frontend component

SettingsPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Load/save error banner.

Virly · Frontend component

SettingsPage

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

"Personal details updated." success banner after saving.

Purpose & context

The post-auth account hub. It loads personal details, toggles between a read-only summary and an editable form (reusing the same validators as `PersonalDetailsAuthForm`), saves via the same endpoint, shows the account email + balance, and provides sign-out.

Anatomy

- `PageHeader` ("Settings").
- Details card: header + Edit; `Skeleton` / read `<dl>` / edit `form` (`Fields`, Save/Cancel); error + success banners.
- Side stack: Account card (email, balance) + Session card (Sign out).

Props / API

None.

State & data

- Local state: `details`, `form`, `errors`, `isLoadingDetails`, `isEditingDetails`, `isSavingDetails`, `successMessage`.
- Hooks: `useAuth`, `useCurrency`, `useNavigate`, `useEffect`, `useState`.
- Data: `api.personalDetails` (`GET /api/accounts/personal-details`), `api.updatePersonalDetails` (`PUT /api/accounts/personal-details`).

Interactions & events

Edit → populate form; Save → validate → update → patch session + success; Cancel → revert; Sign out → `auth.logout` → `/login`.

States & variants

- `default` (read), edit, `loading`, `error`, `success`, saving (button disabled). Empty/disabled: fields show “Not provided” when empty.

Dependencies

- Children: `Primitives` (`Button`, `Card`, `ErrorBanner`, `Field`, `PageHeader`, `PageStack`, `ResponsiveGrid`, `Skeleton`, `SuccessBanner`).
- Helpers: `validateRequiredText`, `validateDateOfBirth`, `useCurrency`.

Accessibility

Fields are labelled (`Field`); banners carry `role`. Read mode uses `<dl>`.

Usage example

```
<Route path="/settings" element={<SettingsPage />} />
```

Related / used by

Routed inside the protected shell; reached from the bottom-pinned sidebar item. Shares validators + endpoint with `PersonalDetailsAuthForm`.

Notes / gotchas

Does not fit the seven fixed feature areas; placed in Shared UI per the rules and flagged for human review in `_inventory.md`. A human may prefer to relocate it (e.g. to Dashboard / Balance as an “account” surface).

ShaderBackground

- **Path:** `client/src/components/ui/shader-background.tsx`
- **Category:** primitive/ui | **Feature area:** Shared UI | **Tier:** Lite
- **Summary:** A full-viewport WebGL plasma/grid background, mounted globally by `App`, that pauses on hidden tabs and respects reduced-motion.

Screenshot(s)

Virly · Frontend component

ShaderBackground

state: default

PLACEHOLDER — replace via Storybook capture (1440x900)

Animated WebGL background behind the app content layer.

Props / API

None.

States & variants

- `default` (animating). Paused when the tab is hidden or the user prefers reduced motion (renders a single static frame). Falls back silently if WebGL is unavailable. Loading/empty/error/success/disabled: N/A.

Usage example

```
// App.tsx
<ShaderBackground />
```

Note: Categorised as a `primitive/ui` (decorative) so it is Lite, but it carries real behaviour (WebGL lifecycle, visibility + reduced-motion handling) — see the source for details.

VideoSessionPage

- **Path:** `client/src/features/video/VideoSessionPage.tsx`

- **Category:** page | **Feature area:** Shared UI (does not fit the seven fixed areas) | **Tier:** Full
- **Summary:** Customer-facing page to start/join a secure support or sales video session. (No matching fixed area; placed here per the rules.)

Screenshot(s)



Start panel (topic + support/sales) beside the meeting stage.

Virly · Frontend component

VideoSessionPage

state: loading

PLACEHOLDER — replace via Storybook capture (1440x900)

Busy while creating/joining a session.

Virly · Frontend component

VideoSessionPage

state: success

PLACEHOLDER — replace via Storybook capture (1440x900)

Active meeting in the stage (JitsiMeeting).

Virly · Frontend component

VideoSessionPage

state: error

PLACEHOLDER — replace via Storybook capture (1440x900)

Unable to start/open the session.

Purpose & context

Lets a customer start a support or sales video call (or resume one via `?sessionId=`). It creates the session, fetches a join token, and renders the meeting. The intro copy explicitly states that video can guide the user but **money movement still requires the normal in-app confirmation flow** — agents cannot move money through the call.

Anatomy

- `PageHeader` + status pill.
- Control panel: secure-session intro, optional topic `TextAreaField`, support/sales choice buttons.
- Stage card: `JitsiMeeting` when active, else an empty stage with status copy.

Props / API

None. (Reads `?sessionId` from `useSearchParams`.)

State & data

- Local state: `session`, `jitsi`, `topic`, `error`, `isBusy`.
- Hooks: `useAuth`, `useSearchParams`, `useCallback`, `useEffect`, `useMemo`, `useState`.
- Data: `api.createVideoSession` (`POST /api/video-sessions`), `api.videoSession` (`GET /api/video-sessions/:id`), `api.videoJoinToken` (`POST .../:id/join-token`), `api.endVideoSession` (`POST .../:id/end`).

Interactions & events

Start (support/sales) → create + join → show meeting; End → end session; `?sessionId` → load + join an existing session.

States & variants

- `default` (ready), `loading` (`isBusy`), `success` (active meeting), `error`, terminal-status empty stages. Disabled: choice buttons while busy.

Dependencies

- Children: `JitsiMeeting`, `Primitives` (`Button`, `Card`, `ErrorBanner`, `PageHeader`, `PageStack`, `ResponsiveGrid`, `TextareaField`), `lucide-react`.

Accessibility

`PageHeader` heading + status pill; the topic field is labelled and hints against sharing secrets; choice buttons are real buttons.

Usage example

```
<Route path="/video" element={<Suspense fallback={<RouteFallback />}  
><VideoSessionPage /></Suspense>} />
```

Related / used by

Lazy-routed by `App`; reachable from the “Video” nav item and the assistant's `video_session_cta`.
Agent side: `AgentVideoSessionsPage`.

Notes / gotchas

Does not fit the seven fixed areas; placed in Shared UI per the rules and flagged for human review.
The topic hint warns users not to share passwords or card numbers on the call.